

TokEval: A Tokenizer Evaluation Suite

Clara Meister

EPFL

clara.meister@epfl.ch

Abstract

Language model tokenizers are typically selected with minimal evaluation, despite the fact that their design choices directly impact model capabilities. This can be partly attributed to a limited understanding of which tokenizer properties affect which aspects of downstream performance. We introduce TokEval, a framework of tokenizer evaluation metrics that goes beyond standard measures like fertility and compression rate to capture linguistically and structurally meaningful properties, e.g., UTF-8 character boundary integrity and digit place-value boundary alignment for mathematics. To validate whether these metrics are predictive of downstream model performance, we conduct controlled language model pretraining experiments, varying solely the tokenizers’ training data mixture, pretokenization strategy, and training algorithm. We evaluate the resulting models on bits-per-byte (a tokenizer-agnostic version of perplexity) and several benchmarks, spanning linguistic understanding, mathematical reasoning, and code generation. Our experiments suggest that different intrinsic properties have different impacts on model abilities: information-theoretic metrics predict language modeling abilities (Spearman $|\rho|$ up to 0.80), while structure-sensitive metrics, such as those measuring digit and line-break handling, correlate with task accuracy. We hope TokEval enables more principled tokenizer evaluation, replacing pretraining sweeps with intrinsic measurement wherever the two agree.

 [cmeister/tokenizer-intrinsic-evals](https://github.com/cmeister/tokenizer-intrinsic-evals)

huggingface.co/cmeister/tokenizer-lm-ablations

1 Introduction

Subword tokenization is a ubiquitous component of modern language model pipelines. Nearly all contemporary large language models (LLMs) rely on some variant of subword segmentation (e.g., Byte Pair Encoding (BPE; Sennrich et al., 2016) or the UnigramLM algorithm (Kudo, 2018)) to map raw text to discrete token sequences, the format on which models can be efficiently trained and evaluated. Despite this central role, tokenizer design often receives limited attention. In contrast to the substantial efforts devoted to optimizing model architectures, training data, and learning algorithms, the tokenizer is typically selected using a small set of heuristics, such as vocabulary size, compression rate, or informal inspection of segmentation quality. The choice is then rarely revisited over the course of model development. This neglect persists despite growing evidence that tokenization choices shape downstream model behavior across various domains, including arithmetic accuracy, multilingual performance, and general task quality (Ali et al., 2024; Singh & Strouse, 2024; Liu et al., 2025; Rust et al., 2021; Lesci et al., 2025). Taken together, these results show that tokenizer design is a substantive modeling choice with measurable consequences for downstream model capabilities.

Perhaps a core reason for this lack of consideration is that testing out tokenizers is an expensive endeavor: the entire pipeline, from text processing to output formatting, must be reconfigured for each different design choice. Furthermore, there are two clear issues with the current intrinsic evaluation protocols:¹ Firstly, the current set of intrinsic evaluation

¹Intrinsic evaluation refers to evaluating a tokenizer in isolation, using only the tokenizer and a corpus, without training a downstream model.

metrics is very narrow. For example, compression rate tells us only *how much* a tokenizer compresses, but not *how well* it preserves the structural properties that matter for specific downstream tasks. Second, the relationship between intrinsic metrics and downstream performance remains contested. As concrete examples, Zouhar et al. (2023) found that Rényi efficiency correlates strongly with machine translation quality, and compression rate has long been treated as the default proxy for tokenizer quality (Rust et al., 2021). Subsequent work, however, directly challenged both findings (Schmidt et al., 2024; Cognetta et al., 2024). We argue that these mixed results suggest not that intrinsic evaluation is hopeless, but rather that we need richer intrinsic metrics that capture domain-specific structural tokenizer properties, and controlled experiments, isolating specific impacts of tokenizer characteristics.

In this paper, we address both needs. Our contributions are as follows:

- We introduce TOKVAL, an open-source library implementing a suite of intrinsic tokenizer evaluation metrics spanning text, math, code, and multilingual fairness, alongside tokenizer visualizations and integrity checks (§3).
- We propose novel intrinsic metrics for mathematical content and code that target specific downstream capabilities (§3.5–§3.6).
- We conduct controlled pretraining experiments across a grid of tokenizer configurations, varying algorithm, pretokenization strategy, and training data mixture while holding all other variables constant (§5)
- We use mixed-effects regression to isolate the contribution of individual metrics while controlling for covariation in the design axes, showing that specific intrinsic metrics are predictive of specific downstream abilities (§5).

2 Background and Related Work

Tokenization Algorithms. To enable a machine learning model to process text, the text must first be represented as a sequence of real-valued vectors. Tokenization is a key step in this preprocessing pipeline. Formally, tokenization maps an input sequence of symbols (typically raw text represented at the byte or character level) to an output sequence of tokens drawn from a finite vocabulary. Each token can then be associated with a real-valued vector, for instance, through an embedding lookup table. Notably, bytes or characters themselves could be used as the vocabulary symbols, in which case, the mapping is trivial. However, byte- and character-level tokenization leads to substantially longer input sequences, which incurs significant computational overhead in both training and inference (Xue et al., 2022; Tay et al., 2022). Subword tokenization, where the vocabulary consists of units whose granularity spans between characters and full words, is the dominant approach, as it provides a favorable tradeoff between vocabulary coverage and sequence length. It both provides a useful inductive bias for learning and improves training and inference efficiency. Byte Pair Encoding (BPE; Sennrich et al., 2016) is the most widely-used subword tokenization algorithm. While various other algorithms have been proposed and taken different approaches (e.g., UnigramLM; Kudo, 2018), many have built on BPE, proposing various modifications to address empirically observed issues, e.g., SuperBPE, BoundlessBPE and PickyBPE (Liu et al., 2025; Schmidt et al., 2025; Chizhov et al., 2024).

Impacts of Tokenization. Several studies have investigated the relationship between tokenization choices and model capabilities. Ali et al. (2024) trained language models, ablating only tokenizer settings, and found that tokenizer choice significantly impacts both LLM downstream performance and training cost. Notably, they saw that standard intrinsic metrics like fertility and parity were not reliably predictive of downstream performance. Rust et al. (2021) similarly performed tokenizer ablations for LLM training and observed that replacing a multilingual tokenizer with a dedicated monolingual tokenizer improves downstream performance across nearly all tasks and languages, with effect size comparable to that of a language’s share of data in the pretraining corpus. Singh & Strouse (2024) demonstrated that digit tokenization directly impacts models’ arithmetic capabilities, observing a $\sim 20\%$ difference in mathematics task accuracy from changing tokenization strategy alone. At a more fundamental level, Lesci et al. (2025) showed *causally* that including a subword in the vocabulary can increase the probability assigned to the corresponding string by up to $17\times$, in comparison to if the string were represented by

two tokens. Pawar et al. (2025) similarly showed that the splitting of natural words into multiple tokens negatively impacts model performance. Tokenizers have also been linked to discrepancies in model cost and accessibility. Both Petrov et al. (2023) and Ahia et al. (2023) documented systematic cross-lingual unfairness introduced at the tokenization stage, sometimes described as a “token tax” on under-represented languages (Lundin et al., 2026) because of the economic consequences it imparts. More recent work has extended this line of inquiry to investigate dialectal and representational disparities (Kanjirangat et al., 2025).

Intrinsic Tokenizer Evaluation. Of particular interest to this work is the development of intrinsic tokenizer metrics and studies of how such metrics relate to downstream model performance. An *intrinsic* metric evaluates a tokenizer in isolation, without training or querying a downstream model; it is a function of the tokenizer and a text corpus alone. By contrast, an *extrinsic* metric measures the effect of a tokenizer on the performance of a model trained with it (e.g., bits-per-byte, task accuracy). Intrinsic metrics are attractive because they are cheap: evaluating a tokenizer often takes seconds, while training a language model to measure its extrinsic effect can take days or weeks. The most widely used intrinsic metrics are compression-based measures, such as corpus token count and fertility, which quantify the extent to which a tokenizer fragments text. Prior work, however, has shown that compression-based metrics alone are not a reliable objective for tokenizer design. For example, Schmidt et al. (2024) showed that minimizing corpus token count does not reliably improve downstream performance, and Cognetta et al. (2024) constructed tokenizers whose Rényi efficiency can be increased arbitrarily while *degrading* model performance. More recent work has therefore argued for evaluating tokenizers through multiple complementary intrinsic signals rather than through compression alone (Lotz et al., 2025). Another line of work studies tokenizer alignment with morphological units. Arnett & Bergen (2025) introduce MorphScore, which measures the alignment between token boundaries and morpheme boundaries, and show that tokenization quality helps explain some cross-linguistic differences in language-model performance, though morphological boundary alignment itself is not consistently predictive. Relatedly, Soler et al. (2024) show that morphology-preserving segmentations tend to yield better contextualized representations than segmentations that split roots and Kanjirangat et al. (2025) show that tokenization parity is especially predictive of downstream performance on dialectal NLP tasks that rely on syntactic and morphological cues.

3 The TokEval Framework

TokEval computes a suite of intrinsic metrics for a tokenizer over a user-specified text corpus. The library supports textual inputs on a per-language basis for fine-grained cross-lingual analysis. Concretely, let $\mathcal{T}$ be a tokenizer and $\mathcal{C} = \{(\mathbf{x}_i, \ell_i)\}_{i=1}^N$ be a corpus of texts $\mathbf{x}_i$ with language labels ℓ_i . We use $|\mathbf{x}|_u$ to denote the length of the text $\mathbf{x}$ in unit $u \in \{\text{bytes}, \text{chars}, \text{words}, \text{lines}\}$, where `bytes` is the default.² Every text is encoded to a token sequence $\mathbf{t}_i = \mathcal{T}(\mathbf{x}_i)$, where each token t comes from a tokenizer’s vocabulary $\mathcal{V}$. All metrics are reported per language and aggregated globally. Table 1 summarizes the metrics; additional details and design choice justifications are provided in the following subsections.

TokEval is implemented in Python. It natively handles five tokenizer classes: `HuggingFace`, which loads both the `tokenizers` (Rust-accelerated) and `transformers` formats through one class, `SentencePiece` (Kudo & Richardson, 2018), a pre-tokenized-corpus class for input that is already segmented into tokens, and `ScriptBPE/MinGram`, which both use the `SCRIPT`-encoding classes (Land & Arnett, 2025); a `register_tokenizer_class` hook lets users register further tokenizer classes. A unified wrapper ensures identical metric computation regardless of backend. TokEval also includes a “sanity-check” module that runs sixteen deterministic checks per tokenizer, e.g., byte coverage, combining-mark handling, digit handling, roundtrip fidelity, and vocabulary reachability, and aggregates them into one

²The appropriate unit depends on the analysis. Bytes require no language-specific definition of word or character boundaries, but they penalize scripts that have larger per-character UTF-8 encodings (1 byte for Latin, 3-4 for CJK, 4 for emoji). Line counts are appropriate for parallel corpora where each line is a translation, making the count constant across languages and isolating the tokenizer from both linguistic conventions and encoding artifacts.

pass/warn/fail severity per tokenizer. A visualization module generates visualizations of tokenized text and standard plots from a results set, including per-metric bar charts and per-language and faceted breakdowns, across the tokenizers being compared.

3.1 Compression and Information-Theoretic Metrics

The library’s implementation of **Compression Rate** uses a ratio of sums: total length of the corpus in the chosen base unit over total length in tokens. This is done rather than a mean of per-text ratios, avoiding bias toward short texts. **Unigram Entropy** is the Shannon entropy of the empirical token frequency distribution, as computed with respect to the input corpus. **Rényi Efficiency** generalizes this entropy to other entropy orders via a hyperparameter α , recovering Shannon entropy when $\alpha = 1$ (Zouhar et al., 2023). To capture sequential structure, **Bigram Entropy** computes the *conditional* entropy of the next token given the previous one, frequency-weighted by how often each bigram-initial token occurs. This metric is largely inspired by the work of Poelman et al. (2025), although it differs in several ways.³ Low bigram entropy indicates highly predictable sequences, which can signal over-fragmentation, a common symptom when a tokenizer was not trained on the language being evaluated (Poelman et al., 2025). **Trigram Entropy** uses the same setup, albeit conditioning on bigrams instead of single tokens. **Vocabulary Utilization** is the fraction of the total vocabulary entries that occur at least once in the corpus. **Token Length** is the mean number of characters per token over all token occurrences in the corpus. Finally, **Average Token Rank** assigns each vocabulary entry a rank by descending frequency (rank 1 = most frequent) and reports the mean frequency rank over all token occurrences in a given language (Limisiewicz et al., 2023). A low value indicates that a few common tokens account for most occurrences, suggesting poor vocabulary utilization.

3.2 Linguistic Alignment Metrics

Fertility is approximately the inverse of Compression Rate, differing only in aggregation and default choice of u . Cross-lingual variation in fertility is a primary indicator of tokenization-induced unfairness (Petrov et al., 2023): fragmenting low-resource languages into many subwords per word degrades downstream performance (Lesci et al., 2025; Pawar et al., 2025). **MorphScore** (Arnett & Bergen, 2025; Arnett et al., 2025b) measures token–morpheme boundary alignment via precision, recall, and F1 over gold-standard morphological segmentations, covering 70 languages. TokEval integrates MorphScore directly, as a submodule of the fork. By default, it uses the MorphScore library’s recommended settings.

3.3 Multilingual Fairness

Tokenizer Fairness Gini (TFG) measure cross-lingual encoding inequality via a Gini coefficient over per-language tokenization costs $c_\ell = \sum_i |t_i| / \sum_i |x_i|_u$. This metric is best computed with $u = \text{lines}$ on a parallel corpus, so that differences in c_ℓ reflect tokenizer behavior rather than cross-lingual variation in text length or encoding overhead. Values near 1 indicate severe cross-lingual inequality. TokEval additionally provides Lorenz curve visualizations. **Vocabulary-Utilization CoV** takes the coefficient of variation across per-language vocabulary utilization u_ℓ , quantifying differences across languages’ use of the vocabulary. A lower CoV means the vocabulary is used to a similar degree across languages; a higher CoV means usage concentrates in some languages. It is undefined when fewer than two languages are present.

3.4 Encoding Fidelity

A tokenizer is not necessarily lossless: the round-trip reconstruction $\hat{x} = \mathcal{T}^{-1}(\mathcal{T}(x))$ may differ from the original due to Unicode normalization, UNK substitution, or whitespace handling. **Reconstruction Exact Match** and, as a graded complement, **Character Error**

³This definition deviates from Poelman et al. (2025) in several ways: (i) each conditional entropy is normalized by $\log_2$ of the number of distinct successors (next tokens) that the context had in the corpus, i.e., by the maximum entropy achievable by a distribution over that same number of successors; (ii) the aggregation is frequency-weighted rather than an unweighted mean over contexts; (iii) bigram-initial tokens containing punctuation or digits are included; (iv) there is no windowing. The library also computes the reference definition for comparison.

Table 1: Summary of TokEval intrinsic metrics. Notation: $p(t)$ is the relative frequency of vocabulary entry t in the input corpus; n_t is the count of all bigrams in the corpus with t in first position; $n_{t,t'}$ is the count of all trigrams with (t, t') in the first two positions; $\mathcal{A}(c)$ is the set of distinct vocabulary entries observed as successors of the context c ; $\hat{\mathbf{x}} = \mathcal{T}^{-1}(\mathcal{T}(\mathbf{x}))$ is the round-trip reconstruction; $B_{\text{obs}}/B_{\text{ideal}}$ are observed/ideal digit boundary sets (see §3.5); $\mathcal{S}$ is the set of special tokens.

Metric	Formula	Interpretation
Compression & Information-Theoretic (§3.1)		
Compression rate	$\text{CR}_\ell = \sum_{i:\ell_i=\ell} \mathbf{x}_i _u / \sum_{i:\ell_i=\ell} \mathbf{t}_i $	Multiplier by which tokenization compresses. $\uparrow$ = more compression.
Unigram entropy	$H(p(\cdot)) = -\sum_t p(t) \log_2 p(t)$	Uniformity of vocabulary usage. $\uparrow$ = more even.
Vocab. utilization	$ \{t \in \mathcal{V} : p(t) > 0\} / \mathcal{V} $	Fraction of vocabulary entries the corpus uses at least once.
Token length	$\sum_i \mathbf{x}_i _{\text{chars}} / \sum_i \mathbf{t}_i $	Mean characters per token occurrence.
Bigram entropy	$\eta(t) = H(p(\cdot t)) / \log_2 \mathcal{A}(t) $; $\bar{\eta} = \sum_t n_t \eta(t) / \sum_t n_t$	Conditional entropy of the next token given the previous one, divided by the entropy the context’s own observed successor set could carry. $\bar{\eta} \in [0, 1]$; $\downarrow$ = predictable sequences.
Trigram entropy	$\eta(t, t') = H(p(\cdot t, t')) / \log_2 \mathcal{A}(t, t') $; $\bar{\eta}_3 = \sum_{t,t'} n_{t,t'} \eta(t, t') / \sum_{t,t'} n_{t,t'}$	As bigram entropy, with the conditioning context one token longer.
Rényi efficiency	$\bar{H}_\alpha(p(\cdot)) = H_\alpha(p(\cdot)) / \log_2 \mathcal{V} $; $H_\alpha(p(\cdot)) = \frac{1}{1-\alpha} \log_2 (\sum_t p(t)^\alpha)$	Generalized uniformity; computed for $\alpha \in \{1, 2, 2.5, 3\}$, and reported here for $\alpha = 2$. $\uparrow$ = more uniform token usage.
Avg. token rank	$\bar{r}_\ell = \sum_{i:\ell_i=\ell} \sum_j r(t_{ij}) / \sum_{i:\ell_i=\ell} \mathbf{t}_i $	Token frequency rank averaged over all token occurrences, where r ranks by frequency in the corpus. $\downarrow$ = few common tokens dominate.
Linguistic Alignment (§3.2)		
Fertility	$\text{Fert}_\ell = \text{mean}_{i:\ell_i=\ell} \mathbf{t}_i / \mathbf{x}_i _{\text{words}}$	Tokens per word; default $u = \text{words}$ (mean of ratios). 1.0 = one token per word.
MorphScore	Morpheme boundary P/R/F1 (external library)	Token–morpheme alignment across 70 languages.
Multilingual Fairness (§3.3)		
TFG (Gini)	$\text{TFG} = \sum_{i,j} c_i - c_j / 2n^2 \mu$; $c_\ell = \sum_i \mathbf{t}_i / \sum_i \mathbf{x}_i _u$	Cross-lingual inequality of tokenization cost. 0 = perfect parity.
Vocab. util. CoV	$u_\ell = \{\text{used vocab entries in } \ell\} / \mathcal{V} $; $\text{CoV} = \sigma(u_\ell) / \mu(u_\ell)$ over languages ℓ	Cross-lingual dispersion of per-language vocabulary utilization (μ = mean, σ = sample std). $\downarrow$ = similar utilization across languages.
Encoding Fidelity (§3.4)		
Exact match	$\mathbb{1}[\hat{\mathbf{x}} = \mathbf{x}]$, averaged over corpus	Strict round-trip reconstruction rate.
CER	$\text{Lev}(\mathbf{x}, \hat{\mathbf{x}}) / \mathbf{x} _{\text{chars}}$	Character-level Levenshtein distance, normalized; can exceed 1.
UNK rate	$n_{\text{unk}} / n_{\text{total}}$	Fraction of tokens mapped to unknown ID.
UTF-8 completeness	$ \{(i, j) : \text{bytes}(t_{ij}) \text{ is valid UTF-8 and } t_{ij} \notin \mathcal{S}\} / \{(i, j) : t_{ij} \notin \mathcal{S}\} $	Fraction of non-special token occurrences that decode to complete characters. Occurrence-weighted in the corpus.
Char split rate	$ \{c : \text{multi-byte, split}\} / \{c : \text{multi-byte}\} $	Proportion of multi-byte chars whose bytes span multiple tokens.
UTF-8 boundary crossing	Frac. of non-special token occurrences spanning >1 character and leaving one of them incomplete	Frequency of tokens produced by a merge across a character boundary.
Digit Tokenization (§3.5)		
Digit boundary F_1	F_1 of B_{obs} vs. B_{ideal} (right-aligned triples)	Place-value alignment of digit span segmentation.
Digit split variability	$H_{\text{pattern}} = -\sum_p (n_p/N) \log_2 (n_p/N)$	Entropy of splitting patterns for same-length digit spans. $\downarrow$ = consistent.
Operator isolation	Frac. of operator occurrences whose covering tokens carry no non-whitespace character from outside the operator span	Whether operators are fused with adjacent operands. Computed separately on prose, code and math; reported here on the math corpus.
Code Tokenization (§3.6)		
AST boundary alignment	Frac. of AST leaf nodes fully aligned with token boundaries	Per-category (identifier, keyword, operator, literal, delimiter).
Ident. fragmentation	Frac. of identifiers split across >1 token	$\uparrow$ = more fragmentation of programmer-defined names.
Indentation consistency	$\rho(\text{nesting depth, whitespace token count})$	Spearman correlation; $\uparrow$ = monotonic depth signal.

Rate (CER) together characterize round-trip fidelity. CER uses the canonical definition of the metric and can exceed 1 when $\hat{x}$ is substantially longer than x (e.g., byte-fallback expansion). High exact-match failure with near-zero CER indicates minor systematic changes (e.g., normalization); high CER indicates fundamental encoding problems. A **UTF-8 Token Completeness** rate < 1 is not necessarily a defect: byte-fallback mechanisms rely on incomplete byte tokens to represent unseen characters. **Character Split Rate** is stratified and reported by byte width (2-, 3-, 4-byte), since CJK characters and emoji are disproportionately affected. **UTF-8 Boundary Crossing Rate** is the fraction of non-special token occurrences whose bytes span more than one character and leave at least one of those characters incomplete. It captures the frequency of tokens produced by a merge across a character boundary, albeit not including (at least one of) the full characters on either side of the boundary.

3.5 Digit Tokenization Quality

Tokenizer treatment of numbers directly affects arithmetic reasoning (Singh & Strouse, 2024). Motivated by these findings, we propose metrics that operationalize behaviors shown to benefit models’ math processing. **Three-digit Boundary Alignment** evaluates whether digit spans are split at positions consistent with right-aligned grouping into triples: for each contiguous digit span of length d , we define *ideal* boundaries at positions $\{d-3, d-6, \dots\}$ from the left and compute precision, recall, and F_1 against the tokenizer’s observed boundaries. For spans with ≤ 3 digits, both boundary sets are empty; we set $F_1 = 1$ by convention iff the tokenizer keeps the span intact (i.e., $|B_{\text{obs}}| = 0$). Results are bucketed by digit length. **Digit Split Variability** captures the consistency of segmentation: inconsistent segmentation of same-length digit spans may inhibit learning generalizable arithmetic patterns. We group digit spans by length, record the splitting pattern of each occurrence (e.g., $XX|XXX$ vs. $X|XXXX$), and compute the Shannon entropy of the resulting pattern distribution. **Operator Isolation** measures whether operators are dedicated tokens rather than fused with adjacent operands (e.g., $+3$). The operator set is the 24 operators the library scores, in six categories: arithmetic, assignment, logical and bitwise, shift, comparison and ternary. An occurrence counts as isolated when no token covering it carries a non-whitespace character from outside the operator’s own span. The metric is computed on three separate corpora, prose, code and math, and reported per domain; we report math results here. The library also pools the three into an occurrence-weighted micro-average over operator occurrences.

3.6 Code Tokenization Quality

Programming languages have deterministic grammars, so syntactic boundaries can be identified without manual annotation. Parsing source code produces an abstract syntax tree (AST): a tree whose internal nodes represent grammatical constructs (function definitions, loops, assignments) and whose leaf nodes represent the atomic syntactic units that appear in the source text. We propose novel metrics that evaluate whether tokenizers respect these boundaries.⁴ **AST Boundary Alignment** checks whether each AST leaf node’s character span $[s, e)$ coincides with token boundaries.⁵ The library also reports the two one-sided rates separately; the **End-Alignment Rate** is the fraction of AST leaf nodes whose end offset e falls on a token boundary, irrespective of s . The alignment rate is reported per node category and aggregated across the 15 programming languages we measure: Bash, C, C++, C#, Go, Java, JavaScript, Lua, PHP, Python, R, Ruby, Rust, Scala, TypeScript. **Identifier Fragmentation** is a complement to AST Boundary Alignment. It is the occurrence-weighted fraction of programmer-defined identifiers that require more than one token to encode. The related **Tokens per Identifier** reports the mean number of tokens per programmer-defined identifier occurrence rather than the fraction that take more than one. **Indentation Consistency** targets whitespace-sensitive languages (Python and Haskell), computing the Spearman rank correlation between nesting depth and the number of whitespace-only tokens in leading indentation per line. Haskell is absent from all of our code results because its tree-sitter grammar crashes the parser, so the values reported here are Python only.

⁴All parsing and identification is done using the python tree-sitter library.

⁵A node is *fully aligned* if both s and e fall on token boundaries.

4 Experimental Setup

We train 1.27B-parameter language models under 46 tokenizer configurations: 44 tokenizers trained using different configurations, plus two off-the-shelf reference tokenizers (Mistral-Nemo, LLaMA-3). We train two sets of models, differing only in their pretraining data: a natural-language-focused track and math+code-focused track. Within each line of experiments, architecture, model training data, and hyperparameters are held constant; the sole experimental variable is the tokenizer. Every model reported in this paper, the tokenizer each was trained with, and a per-run table of the metrics reported here are released at <https://huggingface.co/cmeister/tokenizer-lm-ablations>.

4.1 Tokenizer Configurations

For the tokenizers that we train, we focus on ablating three axes: algorithm, normalization/pretokenization strategy, and training data composition. Vocabulary size is held constant at $\sim 128\text{K}$ entries. This paper uses different overlapping subsets of tokenizer-model pairs in different analyses. The subsets are built from *tokenizer families*: a family groups tokenizers that share the same value on all three design axes and differ only in a training hyperparameter or preset of that shared configuration. §C.1.1 gives the full definition with examples, and the Fam. column of Table 5 lists each row’s family, so the partition can be checked directly. The **primary panel** (P; $n = 29$) takes one representative per family; it is the set on which the aggregate correlations (Tables 2 and 16) and the per-language regressions (Table 3) are computed. The **extended panel** (E; $n = 39$) contains every member of every family; it is used for the family-mean correlations of §F.2 and the predictor fits of §F.1. The **math+code panel** (M; $n = 20$) holds the custom tokenizers with a math+code model: the 19 primary-panel members that have one, plus one English-only build kept with a training defect flagged (§C.1.1); the math+code columns of Table 2 use only the 19 primary-panel members. The **reference panel** (R; $n = 2$) holds the two off-the-shelf tokenizers, which are excluded from every ranking and aggregate statistic, because their training configurations are largely unknown and cannot be controlled for, and serve instead as held-out prediction targets (§F.1); the one exception is the cross-scale ranking-stability check of §E.3, which measures how a fixed set of models reorders between training scales and therefore includes them, as its table states. More detailed inclusion criteria are provided in §C.1.1 and the panel definitions are stated concisely in Table 4.

Algorithms. We train tokenizers with five algorithms: BPE (Sennrich et al., 2016), UnigramLM (Kudo, 2018), SuperBPE (Liu et al., 2025), parity-aware BPE (Foroutan et al., 2026) and MinGram—a minimalist Unigram-style tokenizer trained using an approximation of EM that more closely mimics inference (Land, 2026). BPE and UnigramLM tokenizers are trained with the HuggingFace `tokenizers` library; the SuperBPE, parity-aware BPE and MinGram tokenizers are trained using the authors’ published codebases. All custom tokenizers are trained on 10 GB of text. For the SuperBPE tokenizers, which use a two-part training strategy, we use either plain BPE or parity-aware BPE for the seed tokenizer, inheriting either 64K or 90K merges from this tokenizer before extending to 128K.

Pretokenization and encoding. Simply put, pretokenization splits text before a vocabulary learning algorithm is applied based on a deterministic set of rules, determining what spans of text can become a token in the learned vocabulary. Consequently, it is a design choice with large influence (Arnett et al., 2025a). The main pretokenization strategies we explore are a minimal punctuation-only baseline, the GPT-4o regex (camelCase splitting, left-aligned 3-digit groups, English contractions), a Claude-inspired regex, and a right-aligned digit variant that groups numbers to place values. In ablations, we try targeted variants of these pretokenization strategies. Further details in §A.1. The majority of tokenizers we train use byte-level encoding, which is standard practice. We also train several tokenizers using SCRIPT byte encoding (Land & Arnett, 2025), which groups characters by Unicode script into blocks before a subword algorithm is applied.

Tokenizer training data compositions. The main suite of custom tokenizers are trained on one of three mixtures: English-only, balanced multilingual (35% English, 30% multilingual across 30 languages, 15% math, 15% code), or code-heavy (50% English, 50% code). We sample English training data from FineWeb-Edu (Penedo et al., 2024), multilingual data

from FineWeb2 (Penedo et al., 2025), math data from FineMath (Lozhkov et al., 2024), and code data from StarCoderData (Li et al., 2023). The multilingual portion spans 30 languages across 11 writing systems (Latin, Cyrillic, Arabic, Devanagari, Bengali, Tamil, Thai, Hangul, CJK, Greek, Hebrew) and 11+ language families. Per-language data is sampled proportionally to presence in the original FineWeb2 dataset. Additional tokenizers are trained for ablations, using configurations that weight the 30 multilingual languages equally rather than proportionally, and that restrict the tokenizer’s multilingual training data to only a subset of languages. More details can be found in §A.2.

4.2 Language Model Training

All models use the nanochat architecture (Karpathy, 2025), a decoder-only transformer with 24 layers. The model has a total of 1.27B parameters, of which 682M are in transformer weight matrices. For the *natural-language-focused models*, we use the same datasets as for tokenizer training, albeit in different proportions. The token budget follows nanochat’s compute-optimal scaling rules. Because generative math and code abilities do not surface under the above natural-language-focused mixture at this model scale, we additionally train *math+code models* from scratch on a ~ 20 B-token mixture, 50% math and 50% code by text bytes: math text from MegaMath-Web-Pro and code from The Stack v2’s educational subset. Due to compute constraints, this second track covers a subset of the tokenizers rather than all of them: the math+code panel (M) has 20 custom tokenizers, which with the 2 off-the-shelf references gives the 22 models of Table 18.⁶ Further math+code models outside that panel are used in the appendix analyses (the 24-member extended math+code roster of §F.3, the byte-alphabet retrains, and the whitespace control of Table 14). Full architectural details and hyperparameter values are in §B; training data details are in §B.3.

4.3 Evaluation

Downstream benchmarks and perplexity evaluations. For the natural-language-focused models, we consider bits-per-byte (BPB) on FLORES+ (NLLB Team et al., 2024), over all 215 FLORES+ languages and, separately, restricted to the 31 languages present in our training data. We also evaluate on two minimal-pair agreement benchmarks: BLiMP (Warstadt et al., 2020) and MultiBLiMP (Jumelet et al., 2026) (English and multilingual, respectively). Both are loglikelihood-scored under the BOS-prefixed convention described in §C. Performance on Belebele (four-option multilingual reading comprehension) was at chance, so we do not report those results. The math+code models (§4.2) are evaluated on GSM8K (Cobbe et al., 2021), HumanEval (Chen et al., 2021), and on MBPP (Austin et al., 2021), as well as on bits-per-byte held-out StarCoderData across 7 programming languages (Code BPB), using data disjoint from the training mixture. Generation tasks prepend the BOS token to the prompt, stop at the model’s EOS token, and heal the prompt boundary; shot counts, scoring, the healing procedure, and the BPB computation are in §C.

Intrinsic–downstream correlation. We measure the relationship between intrinsic tokenizer properties and downstream model quality via two complementary analyses. *Aggregate correlation* computes one global intrinsic metric score and one downstream metric score per tokenizer, reporting the Spearman rank correlation between these values. For the intrinsic metrics, this means intrinsic scores are averaged across languages, where applicable. For most downstream metrics, it is already the case that each model receives only a single value, but otherwise (e.g., in the case of FLORES BPB), we likewise average across languages. *Mixed-effects regression* fits $D_{T,L} \sim \beta \cdot I_{T,L} + (1 \mid L)$ for each intrinsic metric I and downstream metric D , where T indexes tokenizers and L indexes languages. We reserve this analysis for metrics that provide scores per language. The random intercept per language accounts for the fact that different languages have different baseline metric scores (e.g., languages’ intrinsic difficulties differ, leading to differing baseline BPBs) regardless of tokenizer choice. The intrinsic metric I is standardized (zero mean, unit SD across tokenizers, within language); the downstream metric D is not. This makes the magnitude of β comparable

⁶19 of the 20 custom tokenizers are also members of P. The remaining one, `claude-english-bpe`, is not: the math+code models were trained before it was realized that this tokenizer lacks a full byte alphabet for fallback. It is included only in the held-out-tokenizer code BPB and MBPP pass@1 fits.

Table 2: Aggregate Spearman ρ between intrinsic metrics and downstream performance ($n = 29$ for the natural-language columns; $n = 19$ for the math+code columns). Natural language intrinsic metrics are measured on the 31-language training data subset in FLORES+, while AST alignment and Operator isolation are measured on code and math data, respectively. **Bold** indicates significance, with stars marking the (BH-adjusted) thresholds: * $p_{\text{adj}} < 0.05$, ** $p_{\text{adj}} < 0.01$, *** $p_{\text{adj}} < 0.001$.

Intrinsic Metric	Val BPB	FLORES		Math+code		BLiMP	MultiBLiMP
		trained	all	Code BPB	MBPP		
Rényi eff. ($\alpha=2$)	-0.57*	-0.80***	-0.39	-0.49	0.21	-0.32	0.37
Digit boundary F_1	-0.42	-0.41	-0.67**	-0.62*	0.21	-0.51*	0.32
Ident. fragmentation	0.53*	0.57*	0.36	0.41	-0.21	0.37	-0.45
Operator isolation (math)	0.54*	0.38	0.55*	0.39	0.27	0.32	-0.25
UTF-8 boundary crossing	0.29	0.47*	-0.21	0.45	-0.42	0.27	-0.41
Trigram entropy	-0.44	-0.66**	0.12	-0.45	0.10	-0.05	0.48*
UTF-8 char split	0.36	0.54*	-0.34	0.24	-0.24	0.09	-0.40
Compression rate	-0.32	-0.51*	0.24	-0.12	0.26	-0.14	0.28
Bigram entropy	-0.35	-0.52*	0.00	-0.18	0.04	-0.27	0.35
Vocab utilization	0.14	-0.07	0.61**	0.56	0.07	0.21	0.02
AST alignment	0.31	0.01	0.36	0.16	0.61*	0.10	-0.05
Fertility	0.04	0.22	-0.57*	-0.39	0.09	-0.18	-0.08
Unigram entropy	-0.27	-0.49*	0.07	-0.02	0.25	-0.22	0.25
Gini coefficient	-0.03	0.18	-0.58*	-0.37	0.08	-0.17	-0.03

across metrics. In plain terms, β reports the change in D associated with a one-SD change in the intrinsic metric, holding language constant. As a concrete example, for $\beta = +0.01$: a tokenizer one SD higher on that metric is associated with FLORES BPB 0.01 higher (worse), within a given language. Alongside each β we report a standard error: the expected size of the estimate’s variation if the experiment were repeated. §C.1.2 describes how these standard errors account for the dependence structure of the tokenizer panels. We also report the median within-language Spearman ρ across the 31 languages, as this can be more interpretable. All p -values are adjusted for multiple testing via Benjamini–Hochberg (BH) FDR correction.

5 Results

The results and following discussion focus on the intrinsic metric–downstream performance relationship. The appendix contains material beyond the intrinsic–downstream relationship reported above: full per-tokenizer downstream evaluations and intrinsic-metric tables (§§ D and E), a structural analysis of math+code results (§F.3), and a per-example validation of code structure metrics on five external code models (§F.4).

Aggregate intrinsic–downstream correlations. Table 2 shows Spearman correlations between intrinsic metrics and downstream performance; significant, here and below, means BH-adjusted $p_{\text{adj}} < 0.05$. Five information-theoretic metrics (§3.1) are significant predictors of FLORES BPB on the trained languages, with $|\rho|$ from 0.49 (unigram entropy) to 0.80 (Rényi efficiency, the strongest correlation in the table). Of the 14 intrinsic metrics, only digit boundary F_1 significantly predicts Code BPB ($\rho = -0.62$), and only AST alignment predicts MBPP ($\rho = 0.61$); we attribute the latter correlation to the vocabulary’s line-break handling, which AST alignment partly reflects (§F.3). Digit boundary F_1 is also the only significant BLiMP predictor ($\rho = -0.51$). However, the correlation’s sign coincides with an algorithm split rather than a digit-segmentation gradient: the three UnigramLM tokenizers all have digit boundary $F_1 = 0.483$ and BLiMP among the panel’s highest (0.828 to 0.841), while BPE RightAlign, whose place-value pretokenization gives it the panel’s highest digit boundary F_1 (1.000), has BLiMP in the panel’s mid-range (0.813 to 0.816; Tables 10 and 12). Consistent with this, removing a single UnigramLM tokenizer pushes the digit-boundary- F_1 /BLiMP correlation just above the corrected significance threshold (§F.2).

Dependence on the measurement corpus. Repeating the analysis with intrinsic metrics measured on FineWeb-2/-Edu instead of FLORES+ (Table 16) changes the significance status of 7 of the 98 aggregate correlations. The other 91 values keep their status; their coefficients

still move, several even changing sign, but these values were not significant in either table. Five of the seven values that do change significance status involve a metric whose value depends on the length unit u , which differs between the two tables ($u = \text{lines}$ on FLORES+, $u = \text{bytes}$ on FineWeb). Three of those five belong to the Gini coefficient: its correlation with FLORES(all) BPB is significant and negative under FLORES+ measurement ($\rho = -0.58$) but positive and not significant under FineWeb measurement ($+0.38$). A reversal in the other direction happens as well: Gini becomes a significant predictor of FLORES(trained) BPB ($+0.62$) under FineWeb measurement while it was not under FLORES+ measurement. The difference in units can explain this: FineWeb is not parallel, so its Gini uses a byte-normalized tokenization-cost basis, which mixes per-script byte-length differences into the inequality measure. The line-normalized FLORES+ Gini is arguably the more representative evaluation. UTF-8 boundary crossing and trigram entropy also become non-significant predictors on FLORES(trained) BPB and MultiBLiMP accuracy, respectively, when the intrinsic metrics are measured on FineWeb data, but these metrics' values do not depend on u , so we do not have a good explanation. Otherwise, the correlations discussed above also hold at the extended-panel level and under leave-one-out, with two qualifications reported in §F.2: unigram entropy's correlation with FLORES(trained) BPB falls just short of significance at the extended level, and removing a single tokenizer puts three of the four above the table-wide corrected threshold.

Fertility and FLORES(all) BPB. One surprising result is that fragmentation is associated with opposite directions of FLORES BPB change depending on which languages' BPB is measured. On the 31 trained languages, more fragmentation is associated with worse BPB: fertility's within-language mixed-effects coefficient on FLORES(trained) BPB is $+0.018$ BPB per SD, significant (Table 3). On FLORES(all) BPB, computed mostly on the 184 of 215 FLORES+ languages absent from both the tokenizer's and the model's training data, the association reverses: fertility's aggregate correlation with FLORES(all) BPB is $\rho = -0.57$ and the Gini coefficient's is -0.58 , both significant (Table 2).⁷ In our tokenizer ablations, this pattern repeats: restricting the tokenizer's training languages from the 30-language balanced mixture to the 6 `highres` languages lowers FLORES(all) BPB by 0.0232, and switching the tokenizer's training data from balanced to code-heavy lowers it by 0.0153; both changes narrow the tokenizer's vocabulary coverage of the FLORES+ languages. BPB is normalized by bytes, so fragmentation is not penalized by construction; the reversal instead reflects a mismatch between the vocabulary and the language being scored. On the untrained languages, a vocabulary specialized toward the trained languages falls back to short, frequent byte-level or near-byte-level fragments, and the model's predictions there are high-entropy but calibrated. A broader multilingual vocabulary instead produces longer, rarer multi-byte tokens on the same text, and the model's conditional distribution over these tokens is fit to the trained languages they resemble, not to the language actually being scored. Cross-entropy penalizes a confident wrong prediction more than a hedged, uncertain one, so the specialized vocabulary's byte-level fallback scores lower (better) BPB even though it encodes the text less efficiently. One could therefore argue that FLORES(all) BPB measures byte-fallback robustness on unseen languages, not multilingual encoding quality on the trained languages.

Within-language predictors of FLORES BPB. When controlling for per-language complexity with a random intercept via mixed-effects regression, the predictiveness of several metrics changes in comparison to aggregate correlations (Table 3). For per-language FLORES BPB, the information-theoretic metrics remain highly significant predictors, but UTF-8 character split rate and fertility also become significant. Fertility in particular illustrates the discrepancy that can arise between the two types of analyses. Its within-language coefficient is the largest in magnitude of any metric ($\beta = +0.018$, $\tilde{\rho} = +0.385$), yet its aggregate correlation with FLORES (trained) BPB in Table 2 is not significant. The likely explanation is that languages differ substantially in baseline FLORES BPB: Tamil and Bengali have far higher BPB than German or Spanish, regardless of tokenizer. Within any given language, higher fertility reliably accompanies worse BPB, but the aggregate correlation pools this within-language relationship together with the between-language baseline differences, which makes the within-language association harder to detect with only $n = 29$ tokenizers. Repeating

⁷ β and ρ values are not directly comparable across the two analyses.

Table 3: Relationship between intrinsic tokenizer metrics (FLORES+) and two per-language downstream outcomes (FLORES BPB and MultiBLiMP accuracy, 31 and 24 languages, respectively), over the paper’s primary panel. β : coefficient from mixed-effects regression (units: outcome units (BPB or accuracy) per SD of the metric); $\pm$ values are tokenizer-family-clustered standard errors (§C.1.2). $\tilde{\rho}$: median within-language Spearman correlation. **Bold**: indicates significance (BH-FDR on the family-clustered p -values). * $p_{\text{adj}} < 0.05$, ** $p_{\text{adj}} < 0.01$, *** $p_{\text{adj}} < 0.001$.

Intrinsic Metric	FLORES BPB		MultiBLiMP acc.	
	β	$\tilde{\rho}$	β	$\tilde{\rho}$
Fertility	+0.018*** ± 0.004	+0.385	-0.004 ± 0.002	+0.060
UTF-8 char split	+0.014*** ± 0.002	+0.484	-0.005* ± 0.002	-0.068
Trigram entropy	-0.013*** ± 0.002	-0.463	+0.003* ± 0.001	-0.001
Bigram entropy	-0.011*** ± 0.002	-0.375	+0.003* ± 0.001	+0.017
Compression rate	-0.010*** ± 0.003	-0.385	$+0.002 \pm 0.002$	-0.063
Rényi eff. ($\alpha=2$)	-0.010*** ± 0.001	-0.378	+0.003* ± 0.001	+0.134
Unigram entropy	-0.010*** ± 0.002	-0.326	$+0.002 \pm 0.001$	-0.163
UTF-8 boundary crossing	+0.004* ± 0.001	+0.360	-0.005* ± 0.001	+0.003
Vocab utilization	-0.002 ± 0.004	-0.191	-0.001 ± 0.002	-0.122

this specification with per-language MultiBLiMP accuracy as the outcome, five of the same nine metrics are significant after BH correction, each in the same direction as its FLORES BPB effect.⁸ Of particular interest is that compression rate is not a significant predictor of MultiBLiMP accuracy, consistent with prior findings that compression alone is not a reliable objective for tokenizer quality. The collective practical implication is that language, and the corpus an intrinsic metric is computed on, need to be controlled for in this kind of analysis.

Predicting a held-out tokenizer. Given a tokenizer’s intrinsic metrics, can we predict its downstream performance? To test this, we fit intrinsic-metric predictors for each of four downstream metrics (val BPB, FLORES (trained) BPB, code BPB, MBPP pass@1) on the extended panels (E and M). Cross-validation is leave-one-family-out: each fold holds out one whole family, so no tokenizer is ever predicted by a fit trained on another member of its family (see §C.1.2). We then use these fits to predict the two off-the-shelf reference tokenizers’ models, which were excluded from every fit. Six of the eight resulting predictions (four downstream metrics times two references) fall inside the 90% prediction interval, the range calibrated on the cross-validation errors to contain 90% of held-out observations (§F.1); LLaMA-3 falls outside it on two targets, FLORES (trained) BPB at +2.10 standardized errors and code BPB at -2.01 (prediction error divided by the cross-validated root-mean-squared error). Rank transfers less well than the point prediction: among the 39-tokenizer extended panel plus the inserted reference, the val-BPB fit places LLaMA-3 at predicted rank 1, while its observed val BPB ranks 21st. The practical use of these fits is therefore screening, not selection: a fit predicts a new tokenizer’s BPB to within a wide but calibrated interval before any model is trained, which is enough to discard clearly unsuitable configurations, but it does not order close candidates, so the final choice among them still requires training runs.

6 Discussion and Conclusion

What intrinsic metrics can and cannot predict. Our results support a division of labor among intrinsic metrics. Information-theoretic and compression-based metrics are reliable for optimizing encoding efficiency: five of them are significant predictors of FLORES(trained) BPB (Table 2), making them useful for narrowing the tokenizer search space before committing to pretraining. They are not a sole proxy for task accuracy (Schmidt et al., 2024; Cognetta et al., 2024): the only significant task-accuracy predictors in Table 2 are structure-sensitive quantities, digit boundary F_1 and AST boundary alignment, and even these reflect differences between design-axis groups (algorithm for the BLiMP correlation, line-break

⁸Fertility’s coefficient is not significant when using clustered standard errors (§C.1.2).

handling across pretokenizers for the MBPP correlation; §§ 5 and F.3) rather than gradients within a single design choice.

How to measure. There are three clear takeaways regarding protocol. First, per-language analysis is essential when evaluating over multilingual data: aggregate correlations can mask or even reverse within-language relationships due to baseline differences across languages, as fertility illustrates (Table 3); the same reasoning motivates tokenizer designs that give each language its own sub-vocabulary. Second, the corpus an intrinsic metric is measured on, and its normalization unit, are part of the metric’s definition: the Gini coefficient changes sign between line-normalized FLORES+ and byte-normalized FineWeb measurement (Table 16). Third, what a downstream metric measures depends on the evaluation population, and this must be accounted for when drawing conclusions about tokenizer impacts: FLORES(all) BPB, computed mostly on languages absent from training, measures byte-fallback robustness of the language models rather than multilingual encoding quality on the trained languages.

Limitations. Main results use a single architecture (nanochat, decoder-only transformer) at 1.27B parameters. Tokenizer–performance relationships may differ at larger scales, where models have greater capacity to compensate for suboptimal tokenization, or under different architectures (e.g., encoder-decoder, state-space models). On the other hand, these models are likely too small for certain abilities to surface, implying our conclusions for these settings may be noisy. Our tokenizer ablations leave many configurations untested. The benchmark suite, though spanning perplexity, linguistic acceptability, math, and code, does not cover important capabilities like long-context reasoning, instruction following, or open-ended generation quality. Further, with $n = 29$ tokenizers, the aggregate correlation analysis has limited statistical power, especially for detecting moderate effect sizes. Non-significant correlations should be interpreted as “not detected” rather than “absent.”

Conclusion. We introduced TokEval, an open-source suite of intrinsic tokenizer metrics spanning text, mathematics, code, and multilingual fairness, and validated it with controlled pretraining across tokenizer configurations at two data mixtures. Intrinsic measurement can replace pretraining sweeps where the two agree: information-theoretic metrics screen for encoding efficiency without training a model, and the held-out prediction analysis (§F.1) shows that such fits bound a new tokenizer’s BPB inside a calibrated interval but do not recover its rank among close candidates. Task accuracy is predicted only by structural metrics targeted at the capability in question, and final selection among close candidates still requires training runs. We hope TokEval makes both steps, the intrinsic screening and the targeted structural checks, standard practice in tokenizer development.

Acknowledgments

We thank Tiago Pimentel, Amit Moryossef, and Craig Schmidt for discussions on the metrics presented in this paper, and Tiago Pimentel additionally for detailed feedback on the manuscript. We thank Sander Land for feedback on the design of the TokEval library and for several design improvements to the codebase. The tokenizer language-modeling ablations reported here were carried out as part of Apertus 2 tokenizer development, with compute provided by the Swiss AI Initiative.

Ethics Statement

This work is partly motivated by the observation that tokenization choices disproportionately affect low-resource languages and non-Latin scripts, leading to higher inference costs and degraded performance on already underserved communities (Petrov et al., 2023; Ahia et al., 2023). By providing tools to measure and compare these disparities (e.g., the tokenizer fairness Gini coefficient and per-language fertility), we hope TokEval can help practitioners identify and mitigate such inequities early in model development. We note, however, that our multilingual coverage (31 languages across 11 scripts) still excludes the vast majority of the world’s languages, and our fairness metrics capture only tokenization-level costs, not directly differences in models’ cross-lingual abilities.

References

- Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9904–9923, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.614. URL <https://aclanthology.org/2023.emnlp-main.614/>.
- Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Buschhoff, Charvi Jain, Alexander Weber, Lena Jurkschat, Hammam Abdelwahab, Chelsea John, Pedro Ortiz Suarez, Malte Ostendorff, Samuel Weinbach, Rafet Sifa, Stefan Kesselheim, and Nicolas Flores-Herr. Tokenizer choice for LLM training: Negligible or crucial? In Kevin Duh, Helena Gomez, and Steven Bethard (eds.), *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 3907–3924, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-naacl.247. URL <https://aclanthology.org/2024.findings-naacl.247/>.
- Catherine Arnett and Benjamin Bergen. Why do language models perform worse for morphologically complex languages? In *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 6607–6623, Abu Dhabi, UAE, 2025.
- Catherine Arnett, Tyler A. Chang, Stella Biderman, and Ben Bergen. Explaining and mitigating crosslingual tokenizer inequities. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025a. URL <https://openreview.net/forum?id=0Y7AxxNCYh>.
- Catherine Arnett, Marisa Hudspeth, and Brendan O’Connor. Evaluating morphological alignment of tokenizers in 70 languages. In *Proceedings of the ICML 2025 Tokenization Workshop (TokShop)*, 2025b. URL <https://arxiv.org/abs/2507.06378>.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Ribeiro, Xin Yuan, Wojciech Zaremba, and Ilya Sutskever. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Pavel Chizhov, Catherine Arnett, Elizaveta Korotkova, and Ivan P. Yamshchikov. BPE gets picky: Efficient vocabulary refinement during tokenizer training. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 16587–16604, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.925. URL <https://aclanthology.org/2024.emnlp-main.925/>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Marco Cognition, Vilém Zouhar, Sangwhan Moon, and Naoaki Okazaki. Two counterexamples to tokenization and the noiseless channel. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 16897–16906, Torino, Italia, May 2024. ELRA and ICCL. URL <https://aclanthology.org/2024.lrec-main.1469/>.

- Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Negar Foroutan, Clara Meister, Debjit Paul, Joel Niklaus, Sina Ahmadi, Antoine Bosselut, and Rico Sennrich. Parity-aware byte-pair encoding: Improving cross-lingual fairness in tokenization. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7514–7538, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.342. URL <https://aclanthology.org/2026.acl-long.342/>.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 2023. URL <https://github.com/EleutherAI/lm-evaluation-harness>.
- Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>. GitHub repository/blog post.
- Jaap Jumelet, Leonie Weissweiler, Joakim Nivre, and Arianna Bisazza. MultiBLiMP 1.0: A massively multilingual benchmark of linguistic minimal pairs. *Transactions of the Association for Computational Linguistics*, 14:193–216, 2026. doi: 10.1162/tacl.a.600. URL <https://aclanthology.org/2026.tacl-1.10/>.
- Vani Kanjirangat, Tanja Samardzic, Ljiljana Dolamic, and Fabio Rinaldi. Tokenization and representation biases in multilingual models on dialectal NLP tasks. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 23992–24010, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1224. URL <https://aclanthology.org/2025.emnlp-main.1224/>.
- Andrej Karpathy. nanochat, 2025. URL <https://github.com/karpathy/nanochat>. Open-source repository and model release.
- Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 66–75, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1007. URL <https://aclanthology.org/P18-1007/>.
- Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 66–71, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-2012. URL <https://aclanthology.org/D18-2012/>.
- Sander Land. MinGram: A minimalist unigram tokenizer with high compression and competitive morphological alignment, 2026. URL <https://arxiv.org/abs/2606.27019>.
- Sander Land and Catherine Arnett. BPE stays on SCRIPT: Structured encoding for robust multilingual pretokenization, 2025. URL <https://arxiv.org/abs/2505.24689>.
- Pietro Lesci, Clara Meister, Thomas Hofmann, Andreas Vlachos, and Tiago Pimentel. Causal estimation of tokenisation bias. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28325–28340, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.

- 18653/v1/2025.acl-long.1374. URL <https://aclanthology.org/2025.acl-long.1374/>.
- Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jiawei Li, Jenny Chim, et al. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- Tomasz Limisiewicz, Jiří Balhar, and David Mareček. Tokenization impacts multilingual language modeling: Assessing vocabulary allocation and overlap across languages. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 5661–5681, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.350. URL <https://aclanthology.org/2023.findings-acl.350/>.
- Alisa Liu, Jonathan Hayase, Valentin Hofmann, Sewoong Oh, Noah A Smith, and Yejin Choi. SuperBPE: Space travel for language models. In *Second Conference on Language Modeling (COLM 2025)*, 2025. URL <https://arxiv.org/abs/2503.13423>.
- Jonas F. Lotz, António V. Lopes, Stephan Peitz, Hendra Setiawan, and Leonardo Emili. Beyond text compression: Evaluating tokenizers across scales. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 32155–32173, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1546. URL <https://aclanthology.org/2025.acl-long.1546/>.
- Anton Lozhkov, Loubna Ben Allal, Elie Bakouch, Leandro von Werra, and Thomas Wolf. FineMath: the finest collection of mathematical content. 2024. URL <https://huggingface.co/datasets/HuggingFaceTB/finemath>.
- Jessica M. Lundin, Ada Zhang, Nihal Karim, Hamza Louzan, Guohao Wei, David Ifeoluwa Adelani, and Cody Carroll. The token tax: Systematic bias in multilingual tokenization. In Everlyn Asiko Chimoto, Constantine Lignos, Shamsuddeen Muhammad, Idris Abdulmumin, Clemencia Siro, and David Ifeoluwa Adelani (eds.), *Proceedings of the 7th Workshop on African Natural Language Processing (AfricaNLP 2026)*, pp. 103–112, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-364-7. doi: 10.18653/v1/2026.africanlp-main.10. URL <https://aclanthology.org/2026.africanlp-main.10/>.
- NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, Loic Barrault, Gabriel Mejia Gonzalez, Prangthip Hansanti, John Hoffman, Semarley Jarrett, Kaushik Ram Sadagopan, Dirk Rowe, Shannon Spruit, Chau Tran, Pierre Andrews, Necip Fazil Ayan, Shruti Bhosale, Sergey Edunov, Angela Fan, Cynthia Gao, Vedanuj Goswami, Francisco Guzmán, Philipp Koehn, Alexandre Mourachko, Christophe Ropers, Safiyyah Saleem, Holger Schwenk, and Jeff Wang. Scaling neural machine translation to 200 languages. *Nature*, 630(8018):841–846, Jun 2024. ISSN 1476-4687. doi: 10.1038/s41586-024-07335-x. URL <https://doi.org/10.1038/s41586-024-07335-x>.
- Sachin Pawar, Manoj Apte, Kshitij Jadhav, Girish Keshav Palshikar, and Nitin Ramrakhiani. Broken words, broken performance: Effect of tokenization on performance of LLMs. In Kentaro Inui, Sakriani Sakti, Haofen Wang, Derek F. Wong, Pushpak Bhattacharyya, Biplab Banerjee, Asif Ekbal, Tanmoy Chakraborty, and Dharendra Pratap Singh (eds.), *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pp. 372–385, Mumbai, India, December 2025. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics. ISBN 979-8-89176-299-2. doi: 10.18653/v1/2025.ijcnlp-short.31. URL <https://aclanthology.org/2025.ijcnlp-short.31/>.

- Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. Fineweb2: One pipeline to scale them all – adapting pre-training data processing to every language, 2025. URL <https://arxiv.org/abs/2506.20920>.
- Aleksandar Petrov, Emanuele La Malfa, Philip Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 36963–36990. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/74bb24dca8334adce292883b4b651eda-Paper-Conference.pdf.
- Wessel Poelman, Thomas Bauwens, and Miryam de Lhoneux. Confounding factors in relating model performance to morphology. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, November 2025. URL <https://arxiv.org/abs/2511.01380>.
- Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? On the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 3118–3135, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.243. URL <https://aclanthology.org/2021.acl-long.243/>.
- Craig W Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. Tokenization is more than compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 678–702, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.40. URL <https://aclanthology.org/2024.emnlp-main.40/>.
- Craig W Schmidt, Varshini Reddy, Chris Tanner, and Yuval Pinter. Boundless byte pair encoding: Breaking the pre-tokenization barrier. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=oPAjXGV8qQ>.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL <https://aclanthology.org/P16-1162/>.
- Aaditya K. Singh and DJ Strouse. Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs. *arXiv preprint arXiv:2402.14903*, 2024.
- Aina Garí Soler, Matthieu Labeau, and Chloé Clavel. The impact of word splitting on the semantic content of contextualized word representations. *Transactions of the Association for Computational Linguistics*, 12:299–320, 2024. doi: 10.1162/tacl_a_00647. URL <https://aclanthology.org/2024.tacl-1.17/>.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2023.127063>. URL <https://www.sciencedirect.com/science/article/pii/S0925231223011864>.
- Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=JtBRnr1OEFN>.

- Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. Blimp: The benchmark of linguistic minimal pairs for english. *Transactions of the Association for Computational Linguistics*, 8:377–392, 2020. doi: 10.1162/tacl_a_00321.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022. doi: 10.1162/tacl_a_00461. URL <https://aclanthology.org/2022.tacl-1.17/>.
- Ge Yang, Edward Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tuning large neural networks via zero-shot hyperparameter transfer. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 17084–17097. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/8df7c2e3c3c3be098ef7b382bd2c37ba-Paper.pdf.
- Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5184–5207, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.284. URL <https://aclanthology.org/2023.acl-long.284/>.

A Tokenizer Training and Configuration Details

A.1 Pretokenization Configurations

Before the tokenization algorithm (BPE, UnigramLM, etc.) is applied, raw text is split into coarse segments called *pretokens* using a deterministic rule. The tokenization algorithm then operates independently within each pretoken: merges (in BPE) or segmentation candidates (in UnigramLM) cannot cross pretoken boundaries. Pretokenization thus defines the maximum granularity of the learned vocabulary; a token can be at most one pretoken long. We look at several pretokenization strategies in this work, described below.

Punctuation. This pretokenizer is defined using the HuggingFace pretokenizer class. It is instantiated as: `Sequence([Punctuation(behavior="Isolated"), ByteLevel(use_regex=True)])`. It isolates punctuation characters, otherwise relying on the byte-level encoder’s default regex (GPT-2) for other split points.

GPT-4o. CamelCase-aware splitting, left-aligned 3-digit groups (`\p{N}{1,3}`), English contractions (`'s`, `'t`, `'re`, etc.), preceding-punctuation attachment.

Claude. Differs from GPT-4o in: whitespace separated by type (spaces, tabs, newlines independently), no punctuation attachment to preceding words, supports both straight and curly apostrophes. We note that this is an approximation of the Claude pretokenization regex, derived using token counts from the Claude API. We do not know definitively the pretokenization choices that Claude uses.

Right-aligned digits. Pretokenizes digit spans to groups of three according to place values (e.g., `"1234567" → ["1", "234", "567"]`). Identical to GPT-4o except the digit group regex uses a lookahead for place-value alignment: `\p{N}{1,3}(?=(?:\p{N}{3})*(?:\p{N}$))`. This ensures `"123456"` tokenizes as `["123", "456"]` (thousands-aligned) rather than left-aligned `["12", "345", "6"]`.

Clean, plus2, plus3. `Clean` is a fifth, multilingual-oriented regex, distinct from the four above: it does not split camelCase, does not explicitly attach contractions, and treats each digit as its own pretoken rather than grouping digits into runs. `plus2` extends `clean` by allowing leading apostrophes (ASCII or curly) to attach to the start of a pretoken, targeting

Panel	<i>n</i>	Membership	Used for
P (primary)	29	One representative per tokenizer family	Aggregate correlations (Tables 2 and 16); per-language regressions (Table 3)
E (extended)	39	Every member of every family	Family-mean correlations (§F.2); predictor fits (§F.1)
M (math+code)	20	Custom tokenizers with a math+code model: 19 are also primary-panel members (§C.1.1)	Math+code benchmarks and correlations (§F.3); the math+code columns of Table 2 use the 19 primary-panel members only
R (reference)	2	The two off-the-shelf tokenizers	Held-out prediction targets (§F.1); excluded from every ranking and aggregate statistic

Table 4: The four tokenizer panels. A tokenizer family groups configurations that share all three design-axis values and differ only in a training hyperparameter or preset; the Fam. column of Table 5 lists each row’s family.

French/Italian/Catalan contractions (e.g., *l’arbre*); `plus3` additionally attaches a trailing apostrophe not followed by a letter, targeting Maltese *ta’*.

A.2 Training Data: Ablation Configurations

The intermediate-language-coverage ablations restrict the tokenizer’s training data to 5 of the 30 multilingual languages plus English (`highres`: `rus`, `spa`, `deu`, `fra`, `cmn`, plus English; 6 languages total) or 20 of the 30 plus English (`highmid`: those same 5 `highres` languages plus 15 mid-resource languages, dropping the 10 lowest-resource languages, plus English; 21 languages total), so the language model still sees all 30 languages during training but the tokenizer never saw the omitted ones. This meant to test impact on models’ generalization to languages absent from tokenizer training. We also train a set of 5 tokenizers that weight the 30 languages equally rather than proportionally, one of which omits the repeat-sampling the other four use to fill low-resource tails.

A.3 Full Tokenizer Panel Specification

Table 5: Configurations for every tokenizer named in this paper: the 44 we train plus the 2 off-the-shelf references, the same 46 rows as Table 12. **Pretok**: P=Punctuation, G=GPT-4o, G2=GPT-2, C=Claude, R=Right-aligned, Cl=Clean-multi, S=SCRIPT-encoding, W=whitespace only. **Data**: B=Balanced, E=English, Co=Code, Hr=Highres (6 langs), Hm=Highmid (21 langs), Eq=Equal-weighted multilingual. **Panels**: P=primary std-1B panel member (P, implies membership in the extended std-1B panel too, so P is printed alone), E=extended std-1B panel member not in the primary panel (E), M=math+code panel member (M), R=off-the-shelf reference (R, never a panel member), -=not a member of any panel; a row can show both a std-1B marker and M. **Fam.**: the family index of §C.1.1; rows sharing an index are members of one family, and the 31 indices partition the 39-member extended panel. Rows outside that panel have no family. **Note**: an axis not captured by the other columns, e.g. the SuperBPE stage-1 base, or a property that disqualifies the row from panel membership. The three English-only builds marked “incomplete byte alphabet” represent 224 of 256 byte values (§C.1.1). BPE Claude (eng.) shows M alone: it is excluded from the std-1B panels for that defect but kept in the math+code panel, with the defect flagged rather than dropped, because no defect-free tokenizer for that configuration has a math+code run at this scale (Table 18).

Tokenizer	Algorithm	Pretok	Data	NFC	Panels	Fam.	Note
Mistral-Nemo	BPE	G	–	–	R	–	
LLaMA-3	BPE	–	–	–	R	–	
BPE Punct (bal.)	BPE	P	B	✗	P, M	1	

continued on next page

Tokenizer	Algorithm	Pretok	Data	NFC	Panels	Fam.	Note
<i>(continued from previous page)</i>							
BPE Punct (eng.)	BPE	P	E	✗	P, M	2	incomplete byte alphabet
BPE GPT-4o (bal.)	BPE	G	B	✗	P, M	3	
BPE GPT-4o NFC	BPE	G	B	✓	P, M	4	incomplete byte alphabet full byte alpha- bet
BPE GPT-4o (eng.)	BPE	G	E	✗	–	–	
BPE GPT-4o (eng., full-byte)	BPE	G	E	✗	P, M	5	
BPE GPT-4o (code)	BPE	G	Co	✗	P, M	6	
BPE Claude (bal.)	BPE	C	B	✗	P, M	7	
BPE Claude NFC	BPE	C	B	✓	P, M	8	incomplete byte alphabet
BPE Claude (eng.)	BPE	C	E	✗	M	–	
BPE RightAlign (bal.)	BPE	R	B	✗	P, M	9	
BPE RightAlign NFC	BPE	R	B	✓	P, M	10	
Unigram GPT-4o	Unigram	G	B	✗	P, M	11	
Unigram Claude	Unigram	C	B	✗	P, M	12	
Unigram RightAlign	Unigram	R	B	✗	P, M	13	
SuperBPE GPT-4o (bal.)	SuperBPE	G	B	✗	P, M	14	
PA-BPE NFC	PA-BPE	G	B	✓	P, M	15	
Unigram GPT-4o (highmid)	Unigram	G	Hm	✗	P	16	
Unigram GPT-4o (highres)	Unigram	G	Hr	✗	P	17	
MinGram	MinGram	S	B	✓	P, M	18	
MinGram	MinGram	S	B	✓	E, M	18	SCRIPT enc., splits line breaks
MinGram	MinGram	S	B	✓	E, M	18	
BPE SCRIPT-enc GPT-4o	BPE	S	B	✓	P, M	19	SCRIPT enc., GPT-4o regex
BPE GPT-2 (bal.)	BPE	G2	B	✗	P	20	
BPE GPT-4o (all-eq.)	BPE	G	Eq	✗	P	21	
BPE Claude (all-eq.)	BPE	C	Eq	✗	P	22	
BPE Punct (all-eq.)	BPE	P	Eq	✗	P	23	
BPE GPT-4o NFC (all-eq.)	BPE	G	Eq	✓	P	24	
BPE GPT-4o (all-eq., no-rep.)	BPE	G	Eq	✗	E	21	
BPE GPT-4o (bal., mf50)	BPE	G	B	✗	–	–	no repeat- sampling raised file cap, null control
BPE NFC clean (bal.)	BPE	Cl	B	✓	P, M	25	

continued on next page

Tokenizer	Algorithm	Pretok	Data	NFC	Panels	Fam.	Note
<i>(continued from previous page)</i>							
BPE NFC clean plus2 (bal.)	BPE	Cl	B	✓	E, M	25	plus2 apostrophe
BPE NFC clean plus3 (bal.)	BPE	Cl	B	✓	E, M	25	plus3 apostrophe
SuperBPE PA-BPE GPT-4o	SuperBPE	G	B	✓	E	26	PA-BPE base
SuperBPE PA-BPE GPT-4o t64k	SuperBPE	G	B	✓	E	26	PA-BPE base
SuperBPE clean C2 (bal.)	SuperBPE	Cl	B	✓	P, M	27	PA-BPE base, C2
SuperBPE clean C3 (bal.)	SuperBPE	Cl	B	✓	E, M	27	PA-BPE base, C3
SuperBPE Punct t64k	SuperBPE	P	B	✗	E	28	
PA-BPE NFC GPT-4o HW	PA-BPE	G	B	✓	E	15	hybrid window
PA-BPE NFC clean	PA-BPE	Cl	B	✓	P	29	
Unigram GPT-4o tuned	Unigram	G	B	✗	E	11	tuned hyper-params
BPE GPT-4o (highres)	BPE	G	Hr	✗	P	30	6 langs
BPE GPT-4o (highmid)	BPE	G	Hm	✗	P	31	21 langs
BPE Whitespace (bal.)	BPE	W	B	✗	–	–	control
Unigram Whitespace tuned (bal.)	Unigram	W	B	✗	–	–	control, SentencePiece, tuned

B Model Architecture and Training Details

B.1 Architecture

The model uses the nanochat d24 configuration (Karpathy, 2025): 24 layers, hidden dimension 1536, 12 attention heads (head dimension 128), multi-head attention, context length 2048. Position information uses RoPE (Su et al., 2024) with base frequency $\theta = 100,000$. Activations are squared ReLU (ReLU²). Normalization is parameterless RMSNorm. QK-norm applies a scale of 1.2 to both queries and keys. Logits are soft-capped: $15 \cdot \tanh(\text{logits}/15)$. Attention uses Flash Attention (Dao, 2024) on Hopper GPUs with PyTorch SDPA as fallback.

The sliding window pattern “SSSL” tiles across layers: three layers attend to $\lceil L/4 \rceil$ tokens (rounded to the Flash Attention tile size), followed by one full-context layer. Alternating layers include value embeddings: per-token learned vectors of dimension 128, projected to the full key-value dimension (1536) via a learned linear layer, added to the value tensor through an input-dependent gate. Per-layer learnable scalars control residual stream scaling and skip-connection blending.

Parameter counts. All models use a $\sim 128\text{K}$ vocabulary (Mistral-Nemo differs slightly in vocab size at 131K, although $\sim 1\text{K}$ of these are special tokens). Under 128K vocabulary, the parameter counts for our architectures are: 682M for transformer matrices, 197M for input embeddings, 197M output projection, 197M for value embeddings (12 layers $\times$ 128K $\times$ 128), 74 scalar parameters. This gives us a total of 1.27B parameters. Embedding parameters scale with vocabulary; of the two off-the-shelf tokenizers, LLaMA-3 matches the standard

128,256-token vocabulary exactly (checked directly against the trained model weights: 0 parameter difference), while Mistral-Nemo’s larger 131,072-token vocabulary adds 12,976,128 parameters, 1.0% of the 1.27B total.

B.2 Training Hyperparameters

We follow the training recommendations of the nanochat library. We provide a summary of training details below, but refer the reader to the repository for further details and hyperparameter choice justifications. Training is done on 4 NVIDIA GH200 GPUs in `bfloat16`.

Training budget. For the natural-language-focused models, we train on $10.5 \times N_{\text{scale}}$ tokens where $N_{\text{scale}} = \text{transformer matrices} + \text{output projection} = 879\text{M}$. This ratio is taken directly from nanochat’s scaling law analysis with Kaplan-style parameter counting ($N \propto C^{0.54}$, $D \propto C^{0.49}$).⁹ This gives a target of 9.23B tokens (8,800 steps at batch size $2^{20} = 1,048,576$). This budget was held fixed rather than extended, based on a check on five tokenizers (four custom and LLaMA-3) whose models we continued training for a further 10B tokens on the same mixture, roughly twice the compute-optimal budget: val BPB improved for all five with the cross-tokenizer ranking preserved, but the math and code generation benchmarks stayed near floor (MBPP pass@1 at most 0.070 across the five continued models) with unstable cross-tokenizer rankings. This motivated the separate, dedicated training of the math+code models.

Optimizer. Weight matrices use the Muon optimizer (Jordan et al., 2024); embeddings, projections and scalars use AdamW. Learning rates are transferred across widths via μP (Yang et al., 2021). Base learning rates: 0.02 (Muon), 0.30 (input embeddings), 0.008 (output projection), 0.50 (scalar parameters). AdamW LR’s scaled by $(d_{\text{model}}/768)^{-1} = 0.5$ (μP (Yang et al., 2021)). All LR’s scaled by $\sqrt{B/B_{\text{ref}}} = \sqrt{2} \approx 1.414$ where $B_{\text{ref}} = 524,288$. We use `torch.compile(dynamic=False)` and gradient accumulation of 8 micro-batches. Table 6 lists the resulting effective learning rates and weight decay per parameter group.

Table 6: Effective learning rates and optimizer settings per parameter group.

Group	Optimizer	Eff. LR	Betas	WD
Transformer matrices	Muon	0.0283	mom. schedule	0.050→0
Input embeddings	AdamW	0.2121	(0.8, 0.995)	0.001
Output projection	AdamW	0.0057	(0.8, 0.96)	0.01
Value embeddings	AdamW	0.1061	(0.8, 0.995)	0.01
Residual scalars	AdamW	0.0071	(0.8, 0.95)	0.05
Skip scalars	AdamW	0.707	(0.96, 0.95)	0.0

Schedule. We use a linear LR warm-up over 40 steps, holding it constant until 35% of training. LR then decays linearly over the remaining 65% to 5% of peak. Muon momentum: $0.85 \rightarrow 0.97$ over 400 steps, constant at 0.97, then $0.97 \rightarrow 0.90$ during warmdown. Weight decay: $\lambda_{\text{ref}} \cdot \sqrt{B/B_{\text{ref}}} \cdot (D_{\text{ref}}/D) = 0.050$, with cosine decay to zero (T_{epoch} framework, which scales weight decay to hold $T_{\text{epoch}} = B/(\eta\lambda D)$ constant).

B.3 Training Data Composition

B.3.1 Natural Language-Focused Track

Our model training data consists of 36.9% English web (FineWeb-Edu), 33.4% multilingual in 30 languages spanning 11 scripts (FineWeb2), 16% math (FineMath), and 13.6% code (StarCoderData), where percentages are computed using text bytes. Note that the multilingual portion creates a $266\times$ data scarcity gradient from Russian (18% of bytes) to Tamil (0.1%), mimicking the drastic differences between languages’ representations observed in most real-world datasets. We use temperature sampling to create our dataset, with sampling

⁹See <https://github.com/karpathy/nanochat/blob/master/dev/LOG.md>

weights derived from data proportions in the original dataset. This produces a data scarcity gradient from Russian (18.1% of the multilingual data’s bytes, 531K docs) to Tamil (0.1%, 1.8K docs). Table 7 gives the overall domain composition (English, multilingual, math, code). The full per-language breakdown is in Table 8.

Table 7: LM training data composition (5M documents, 25 GB UTF-8 text).

Domain	Source	Bytes	%
English web	FineWeb-Edu	9.2 GB	36.9
Multilingual (30 langs)	FineWeb2	8.3 GB	33.4
Math	FineMath-4plus	4.0 GB	16.1
Code (Python)	StarCoderData	2.5 GB	10.0
Code (JavaScript)	StarCoderData	0.9 GB	3.6
Total		25.0 GB	100.0

Table 8: Multilingual training data by language, sorted by byte share. 11 writing systems, 11+ language families. Data scarcity spans 266× from Russian to Tamil.

Language	Code	Script	GB	%	Docs
Russian	rus_Cyrl	Cyrillic	4.521	18.1	531,387
Spanish	spa_Latn	Latin	0.423	1.7	122,231
German	deu_Latn	Latin	0.353	1.4	118,340
French	fra_Latn	Latin	0.364	1.5	103,496
Chinese	cmn_Hani	CJK	0.326	1.3	95,805
Japanese	jpn_Jpan	CJK	0.204	0.8	63,316
Italian	ita_Latn	Latin	0.165	0.7	59,676
Portuguese	por_Latn	Latin	0.189	0.8	59,460
Turkish	tur_Latn	Latin	0.152	0.6	47,074
Indonesian	ind_Latn	Latin	0.236	0.9	45,970
Polish	pol_Latn	Latin	0.107	0.4	43,902
Ukrainian	ukr_Cyrl	Cyrillic	0.211	0.8	34,088
Dutch	nld_Latn	Latin	0.071	0.3	32,404
Romanian	ron_Latn	Latin	0.097	0.4	29,203
Arabic	arb_Arab	Arabic	0.168	0.7	27,292
Hungarian	hun_Latn	Latin	0.099	0.4	25,066
Vietnamese	vie_Latn	Latin	0.109	0.4	21,306
Czech	ces_Latn	Latin	0.077	0.3	21,696
Greek	ell_Grek	Greek	0.108	0.4	17,672
Thai	tha_Thai	Thai	0.087	0.3	11,639
Finnish	fin_Latn	Latin	0.034	0.1	11,721
Slovak	slk_Latn	Latin	0.028	0.1	10,015
Bulgarian	bul_Cyrl	Cyrillic	0.052	0.2	9,513
Korean	kor_Hang	Hangul	0.029	0.1	8,263
Croatian	hrv_Latn	Latin	0.026	0.1	8,395
Catalan	cat_Latn	Latin	0.014	0.1	5,087
Hindi	hin_Deva	Devanagari	0.032	0.1	4,928
Hebrew	heb_Hebr	Hebrew	0.025	0.1	4,665
Bengali	ben_Beng	Bengali	0.024	0.1	3,525
Tamil	tam_Taml	Tamil	0.017	0.1	1,831

B.3.2 Math+Code-Focused Track

We build the math+code mixture from two sources: MegaMath-Web-Pro (LLM360 MegaMath; ~50 GB text, ~14M documents) for math, and The Stack v2’s educational subset for code, restricted to seven languages weighted proportionally to their share of the source data: JavaScript (12.7 GB, 3.4M docs), Java (11.0 GB, 3.4M docs), Python (11.0 GB, 2.3M docs), C++ (7.1 GB, 1.4M docs), TypeScript (3.5 GB, 1.3M docs), Go (3.4 GB, 0.8M docs), and Rust (1.3 GB, 0.2M docs), together ~50 GB and ~13M documents. Eight further `stackv2-edu` languages (C, C#, PHP, Ruby, Shell, SQL, Swift, Markdown) are excluded as less relevant to the code benchmarks of §4.3. The combined mixture is ~100 GB of text over ~27M documents, 50% math and 50% code by bytes; documents are sampled with the same weighted-random mechanism as the natural-language mixture above, so any prefix of the mixture keeps this ratio. We use a ~20B-token prefix to train the math+code models.

C Evaluation Details

C.1 Downstream Evaluation Configurations

Loglikelihood scoring. BLiMP and MultiBLiMP are minimal-pair benchmarks: each item pairs an acceptable sentence with a minimally differing unacceptable one, the model assigns each full sentence its total log-likelihood, and the item counts as correct when the acceptable sentence receives the higher total. Accuracy is the fraction of items scored correctly; no generation is involved. Both benchmarks score the sentences from an empty context: the beginning-of-sequence token is prepended and every token of the sentence is scored (the BOS-prefixed convention), matching how documents are packed during training, where every document is itself preceded by the beginning-of-sequence token. The totals are unnormalized log-likelihoods, so sentence pairs whose members tokenize to different lengths are compared on unequal token counts; this is the lm-evaluation-harness reference convention for these tasks, and §E.3 notes one consequence for cross-scale BLiMP comparisons.

Generation harness. GSM8K, HumanEval, and MBPP (§F.3) use the Language Model Evaluation Harness (v0.4.11; Gao et al., 2023): GSM8K is 8-shot chain-of-thought with exact-match scoring on the full 1,319-problem test set; HumanEval is pass@1 under greedy 0-shot decoding; MBPP is pass@1 under greedy 3-shot decoding. Generation heals the prompt boundary: where the prompt’s own tokenization does not end on a token boundary the tokenizer would itself produce there, the decoder drops the affected trailing prompt token(s) and constrains generation until their content is re-emitted, so scoring never starts from a token sequence absent from training.

Bits-per-byte. $BPB = \sum_i NLL_i / (\ln 2 \cdot B)$, where NLL_i is the model’s negative log-likelihood, in nats, for scored token i ; dividing by $\ln 2$ converts nats to bits; and B is the UTF-8 byte length of the decoded scored tokens. Normalizing by byte length rather than by token count makes the metric comparable across tokenizers regardless of differences in vocabulary or compression rates.

C.1.1 Tokenizer Panel Inclusion Criteria

Eligibility. Membership in the primary correlation panel (P) is criteria-defined. A tokenizer is eligible if it meets all four of the following conditions:

- it is trained on the paper’s standard source corpus; any composition, subset, or reweighting of that corpus is a value on the training-data axis, so the language-coverage and reweighting ablations remain eligible;
- its vocabulary falls in the $\sim 128K$ size band;
- it carries no known tokenizer-training defect;
- it is not a deliberate minimal-structure or null-control probe.

Tokenizer families. Two tokenizers belong to one family when they share the same value on all three design axes, algorithm, normalization/pretokenization strategy, and training data composition, and differ only in a training hyperparameter or preset of that shared configuration. For example, BPE GPT-4o (all-eq.) and its no-repeat-sampling variant differ only in a sampling option and form one family, whereas BPE GPT-4o (bal.) and BPE GPT-4o NFC differ in a value on the normalization axis and form two families. Options internal to one algorithm’s implementation also count as presets: the two MinGram builds, which differ only in whether merges may cross line breaks, form one family under this clause. Treating line-break crossing as a value on the pretokenization axis would instead place them in two families. The 31 families partition the 39-member extended panel, and the Fam. column of Table 5 lists every row’s family.

Panel construction. Every member of every family is a member of the **extended** panel ($n = 39$). The **primary** panel takes one representative per family, chosen at the family’s

default preset, except two families, which contribute none.¹⁰ This gives the primary panel 29 members. The primary panel’s members are therefore near-independent design points by construction; strong dependencies between points would violate independence assumptions important to the validity of correlational analyses. The extended panel’s same-family members are not independent, so inference at that level treats families as units; §C.1.2 gives the procedure.

Reference tokenizers. The two off-the-shelf reference tokenizers are held out of both panels. We use them as prediction targets (§5).

Defect exception: three tokenizers without a full byte alphabet. Three of the tokenizers we trained cannot represent all 256 byte values: the English-only builds under the punct, GPT-4o and Claude pretokenizers each represent 224 of the 256, and none has a vocabulary entry containing `{`. The cause is that the BPE trainer seeds its single-byte vocabulary only from byte values that occur in the tokenizer’s own training sample, and merges can only build on entries that exist, so a byte absent from that sample is unrepresentable. This is arguably a violation of the defect-free criterion that we stated earlier as necessary for inclusion, but tokenizers often do not contain the full byte- or character-alphabet (for example, byte-fallback is off by default in the HuggingFace Tokenizers library). We thus did not label these tokenizers as defective. We note that tokenizers and std-1B models were retrained with the full byte alphabet for the GPT-4o and Claude pretokenization builds. A math+code model was also trained for the full-byte GPT-4o variant. The Claude pretokenizer math+code variant is kept in the math+code panel with the defect flagged, as are the std-1B and math+code models for the punct variant. The sensitivity of the published correlations to the punct build, including the one result whose corrected significance depends on it, is quantified in §F.2.

C.1.2 Statistical Testing Protocol

The tokenizer panels are designed grids, not random samples, and two sources of statistical dependence follow from that design. First, members of one tokenizer family are close to duplicates, so treating every tokenizer as an independent observation would overstate the amount of evidence and understate uncertainty; the family partition of §C.1.1 is the unit that addresses this. Second, the per-language analyses observe each tokenizer once per language, so the rows contributed by one tokenizer are correlated with each other; clustering standard errors by tokenizer addresses this. Throughout, a *standard error* is the expected size of an estimate’s variation if the experiment were repeated, tests are two-sided, and significance means a Benjamini–Hochberg-adjusted $p < 0.05$ (§4.3). Table 9 maps each analysis to the panel it runs on and the uncertainty method it uses; the paragraphs below follow the same order.

Primary panel: analytic Spearman tests. On the primary panel, Spearman correlations are tested with the analytic p -value, adjusted within the correction families listed under Multiple comparisons below. §C.1.1 states why the panel’s construction makes the independence approximation reasonable, and §F.2 probes the remaining shared-design-axis structure directly.

¹⁰Both are SuperBPE configurations. SuperBPE trains in two stages: stage 1 trains a seed tokenizer, and stage 2 extends the seed with additional merges under a second pretokenization regex. The stage-2 regex is the regex that segments text when the finished tokenizer is applied, so it is the value the pretokenization axis considers for a SuperBPE configuration; the choice of seed tokenizer, including the regex the seed was trained under, is a training choice we consider internal to the algorithm, on the same footing as a hyperparameter. Each stage-2 regex is therefore represented once, at the default plain-BPE seed and default preset: the plain-BPE-seeded build for the GPT-4o regex, and the C2 build for the clean regex, whose family also holds the C3 preset variant. The two families without a representative differ from an already-represented configuration only in choices internal to the algorithm: the pair seeded from a parity-aware BPE tokenizer under the GPT-4o stage-2 regex, and the punct-seeded variant, which combines a non-default seed with a non-default transition point. Adding a representative from either family would place a second point in a cell of the design grid that is already represented, without ablating any design axis. Both remain their own families in the extended-level analyses, which operate on them as units.

Analysis	Panel (n)	Point estimate	Uncertainty and correction
Aggregate correlation (Tables 2 and 16)	P (29); math+code columns 19	Spearman ρ	Analytic p -value; BH correction within each table's printed cells
Family-mean correlation (§F.2)	E (39 tokenizers in 31 families)	Spearman ρ on family means	Permutation p -value over intact families; BH correction within the artifact grid
Per-language regression (Table 3)	$P \times \text{languages}$	Mixed-effects β	Tokenizer-clustered standard errors; BH correction per outcome
Held-out prediction (§F.1)	Fits on E and M	Predicted downstream value	90% prediction interval calibrated on cross-validation errors

Table 9: The four inference layers, the panel each runs on, and the uncertainty method each uses.

Extended panel: family-mean permutation tests. At the extended level, each family is first averaged to one point before ranking, so same-family tokenizers cannot count as independent evidence. Significance is then assessed with a permutation test: the outcome values are reshuffled among families $B = 10,000$ times with a fixed seed, keeping each family intact, and the permutation p -value is the fraction of reshuffles whose $|\rho|$ reaches the observed value, computed as $(h + 1) / (B + 1)$, where h counts the qualifying reshuffles, so the p -value is never exactly zero. The statistic is not computed below 5 families.

Held-out prediction intervals. The same family partition defines the cross-validation folds of the held-out prediction analysis (§F.1): each fold holds out one whole family, so no fit is scored on a family member of its own training data. A 90% prediction interval is the range expected to contain 90% of new held-out observations; here it is a closed-form t -interval calibrated on these cross-validation errors, not a bootstrap interval.

Per-language regressions: two fits, clustered standard errors. The cells of Table 3 combine two fits. The printed β is the mixed-effects estimate (outcome $\sim$ metric + (1 | language), metric standardized). The printed standard error and the p -value behind the stars come from a second fit of the same within-language estimand: ordinary least squares with language fixed effects and standard errors clustered on the tokenizer, that is, computed while allowing the residuals contributed by one tokenizer to be correlated across the languages it appears in rather than treated as independent ($G = 29$ clusters; each family contributes one tokenizer to this panel, so tokenizer-level and family-level clustering coincide here). The second fit is needed because the mixed-effects standard errors do not account for that correlation; the two fits' coefficients agree at the printed precision.

Clustering increases the standard errors by factors of 1.05 to 2.77 and changes the significance status of one of the 18 metric-outcome cells: fertility against MultiBLiMP accuracy, adjusted $p = 0.00026$ unclustered against 0.063 clustered. Six further cells keep their significance at a weaker star level, five in the MultiBLiMP column and UTF-8 boundary crossing in the FLORES column.

The clustered p -values use a normal reference distribution. Under the more conservative t reference with $G - 1$ degrees of freedom, the FLORES column is unchanged, while in the MultiBLiMP column the adjusted p -values of four of the five significant metrics rise just above the threshold (0.055) and UTF-8 boundary crossing stays significant (0.036). Both inference variants are in the released artifacts.

Multiple comparisons. Benjamini–Hochberg correction is applied within the following correction families:

- the 98 printed cells of Table 2 (14 metrics $\times$ 7 downstream columns);
- the 98 printed cells of Table 16, the parallel grid under FineWeb measurement;

- the 160-cell grid of the released correlation artifact (20 metrics $\times$ 8 downstream targets); this grid is the correction family behind the natural-language code BPB correlations quoted in §F.3, a downstream column computed but not printed in the paper’s tables;
- the 9 metrics of each outcome of Table 3, corrected per outcome;
- the 38 code-structure tests of §F.3;
- the per-example families of §F.4, one per text basis and task, of 7 to 9 tests each.

One association is tested in two of these families on two rosters: AST boundary alignment against MBPP pass@1 is $\rho = 0.61$ over the 19 primary-panel members with a math+code model (Table 2) and $\rho = 0.65$ over the full 20-member math+code panel (§F.3); the two values describe the same relationship on nested rosters. The line-break statistics of §F.3 and the cross-model rank correlation of §E.4 are pre-specified single comparisons rather than scans over a metric grid, and are reported without correction: the former as raw p -values, the latter as a bootstrap confidence interval, a range of values obtained by recomputing the statistic on many resamples of the data, expected to contain the true value in 95% of repetitions.

Robustness and disclosure. Panel membership was frozen before any statistic was computed, and results are reported regardless of direction. Each correlation discussed in §5 has a standing robustness artifact: single-member leave-one-out over the primary panel, plus recomputation with each design-axis group removed in turn (§F.2). Where a leverage check removes members, as in the operator-isolation paragraph of §F.3, the full-panel result is reported first and the removal is a labeled diagnostic, not a substitution.

D Intrinsic Evaluation Results

Tables 10 and 11 report the intrinsic metric values for 43 of the 44 trained tokenizers and for the two off-the-shelf references, measured on FLORES+ and on FineWeb-Edu/FineWeb2 respectively; the two tables’ length-unit conventions differ and are stated in their captions. The one tokenizer of Table 12 absent from both, Unigram Whitespace tuned, has no intrinsic measurement; it is a minimal-structure control and a member of no panel, so nothing correlated or ranked in this paper depends on it.

E Language Model Results

E.1 Language Model Evaluation Results (Full-Scale; 1.27B)

Table 12 reports absolute scores per model on downstream benchmarks for every tokenizer named in this paper, which is a superset of P: it adds the extended-panel-only variants, the two defective English-only retrains, the file-cap null control and the two minimal-structure controls. Table 5 gives each row’s panel membership and configuration.

E.2 Language Model Evaluation Results (Pilot-Scale; 300M)

We also train 300M-parameter models (nanochat d16: 16 layers, 1024 hidden dimension, 8 heads) on the same data; Table 13 reports val BPB, FLORES BPB, and code BPB for this scale. Note that these models cover only a subset of tokenizers from the entire panel, in particular, the ones that were part of the initial panel. These use 3.5B tokens following the same 10.5 $\times$ scaling ratio from the nanochat library. Pilot models serve as a fast iteration loop; between the 300M and 1.27B scales the tokenizer ranking by validation BPB has Kendall $\tau = 0.752$ (§E.3).

E.3 Ranking Stability Across Training Scales

We report Kendall’s τ between per-tokenizer val BPB rank vectors at each pair of the four training scales (d8 $\sim$ 222M, d12 $\sim$ 381M, d16 $\sim$ 596M, d24 1.27B), for the 18-tokenizer panel, in Table 17. All models at each scale were trained under one software configuration for that scale. Adjacent-scale τ is 0.882 (d8-d12), 0.922 (d12-d16), and 0.752 (d16-d24); the

Table 10: Intrinsic tokenizer evaluation results computed on a subset of the FLORES+ validation set, in particular on the subset of languages in our training data. Code and math metrics are computed on StarCoder and math data respectively. Length-unit-dependent metrics use $u = \text{lines}$, with one exception: Fertility keeps its default $u = \text{words}$ in both this table and Table 11 (Table 1), so its two columns differ only by measurement corpus. Compression is therefore measured in *lines* per token and is not comparable with the Compression column of Table 11, which uses $u = \text{bytes}$. Op Isolation is the math-corpus domain, not the value pooled over prose, code and math. Tokenizers are the 43 members of the Table 12 set that have an intrinsic measurement (§D), plus the two off-the-shelf reference tokenizers (marked with [†]), excluded from every ranking and aggregate statistic except the cross-scale check of Table 17.

Tokenizer	Fertility ↓	Compression ↑	Gini ↓	Vocab CoV ↓	$\bar{H}_2$ ↑	Bigram η ↑	Char Split ↓	AST Align ↑	Digit F ₁ ↑	Op Isolation ↑
Mistral-Nemo [†]	4.78	0.0241	0.100	0.435	0.554	0.888	0.008	0.535	0.483	0.987
LLaMA-3 [†]	5.26	0.0185	0.238	0.447	0.505	0.822	0.137	0.511	0.643	0.987
BPE Punct (bal.)	4.78	0.0215	0.196	0.495	0.535	0.843	0.025	1.000	0.662	1.000
BPE Punct (eng.)	9.39	0.0131	0.267	0.721	0.501	0.767	0.247	1.000	0.671	1.000
BPE GPT-4o (bal.)	4.34	0.0235	0.128	0.491	0.561	0.890	0.031	0.545	0.643	0.987
BPE GPT-4o NFC	4.34	0.0236	0.126	0.491	0.561	0.891	0.031	0.545	0.643	0.987
BPE GPT-4o (eng.)	9.57	0.0130	0.269	0.735	0.499	0.772	0.287	0.658	0.643	0.990
BPE GPT-4o (eng., full-byte)	9.57	0.0130	0.269	0.735	0.499	0.772	0.287	0.683	0.643	0.990
BPE GPT-4o (code)	7.59	0.0139	0.256	0.676	0.513	0.780	0.243	0.516	0.643	0.987
BPE Claude (bal.)	4.33	0.0237	0.125	0.486	0.560	0.889	0.030	0.748	0.643	0.997
BPE Claude NFC	4.32	0.0237	0.123	0.486	0.560	0.890	0.030	0.748	0.643	0.997
BPE Claude (eng.)	9.41	0.0132	0.264	0.734	0.504	0.774	0.274	0.787	0.643	0.997
BPE RightAlign (bal.)	4.34	0.0235	0.128	0.487	0.561	0.889	0.031	0.545	1.000	0.987
BPE RightAlign NFC	4.34	0.0236	0.126	0.487	0.561	0.890	0.031	0.545	1.000	0.987
Unigram GPT-4o	4.27	0.0192	0.121	0.393	0.324	0.770	0.049	0.724	0.483	1.000
Unigram Claude	4.30	0.0192	0.120	0.380	0.333	0.773	0.054	0.794	0.483	1.000
Unigram RightAlign	4.27	0.0192	0.121	0.394	0.324	0.770	0.049	0.724	0.483	1.000
SuperBPE GPT-4o (bal.)	4.40	0.0234	0.158	0.541	0.629	0.899	0.037	0.416	0.643	0.808
PA-BPE NFC GPT-4o	3.99	0.0275	0.017	0.259	0.540	0.911	0.015	0.605	0.633	0.990
Unigram GPT-4o (highmid)	4.81	0.0161	0.224	0.492	0.340	0.740	0.186	0.724	0.483	1.000
Unigram GPT-4o (highres)	6.48	0.0141	0.201	0.618	0.335	0.724	0.221	0.716	0.483	1.000
MinGram SCRIPT-enc	4.22	0.0244	0.119	0.478	0.542	0.891	0.001	0.748	0.637	0.997
MinGram SCRIPT-enc (nl-split)	4.22	0.0244	0.119	0.478	0.542	0.891	0.001	0.748	0.637	0.997
BPE SCRIPT-enc GPT-4o	4.31	0.0237	0.123	0.490	0.557	0.890	0.001	0.546	0.643	0.987
BPE GPT-2 (bal.)	4.76	0.0216	0.195	0.493	0.546	0.848	0.026	0.749	0.662	0.997
BPE GPT-4o (all-eq.)	4.07	0.0264	0.050	0.267	0.547	0.910	0.015	0.546	0.643	0.987
BPE Claude (all-eq.)	4.06	0.0265	0.050	0.259	0.545	0.909	0.015	0.748	0.643	0.997
BPE Punct (all-eq.)	4.60	0.0230	0.166	0.343	0.528	0.842	0.014	1.000	0.660	1.000
BPE GPT-4o NFC (all-eq.)	4.07	0.0265	0.048	0.267	0.547	0.912	0.015	0.546	0.643	0.987
BPE GPT-4o (all-eq., no-rep.)	4.07	0.0264	0.050	0.267	0.547	0.910	0.015	0.546	0.643	0.987
BPE GPT-4o (bal., m50)	4.34	0.0235	0.128	0.491	0.561	0.890	0.031	0.545	0.643	0.987
BPE NFC clean (bal.)	4.33	0.0234	0.137	0.507	0.557	0.881	0.032	0.749	0.483	0.997
BPE NFC clean plus2 (bal.)	4.35	0.0235	0.122	0.498	0.557	0.883	0.030	0.749	0.483	0.997
BPE NFC clean plus3 (bal.)	4.35	0.0235	0.122	0.499	0.557	0.883	0.030	0.751	0.483	0.997
SuperBPE PA-BPE GPT-4o	3.99	0.0277	0.036	0.313	0.598	0.926	0.016	0.446	0.643	0.845
SuperBPE PA-BPE GPT-4o t64k	4.02	0.0271	0.051	0.355	0.610	0.926	0.018	0.425	0.643	0.815
SuperBPE clean C2 (bal.)	4.09	0.0269	0.036	0.328	0.545	0.908	0.016	0.734	0.483	0.997
SuperBPE clean C3 (bal.)	3.97	0.0277	0.040	0.326	0.615	0.928	0.016	0.517	0.643	0.838
SuperBPE Punct t64k	4.47	0.0232	0.169	0.535	0.658	0.898	0.031	0.445	0.650	0.391
PA-BPE NFC GPT-4o HW	3.95	0.0274	0.030	0.252	0.542	0.913	0.014	0.566	0.643	0.990
PA-BPE NFC clean	4.12	0.0266	0.019	0.276	0.540	0.898	0.015	0.752	0.483	0.997
Unigram GPT-4o tuned	4.19	0.0192	0.125	0.402	0.327	0.776	0.051	0.721	0.483	0.997
BPE GPT-4o (highres)	7.52	0.0143	0.309	0.656	0.484	0.772	0.268	0.543	0.643	0.987
BPE GPT-4o (highmid)	5.07	0.0180	0.302	0.526	0.470	0.807	0.205	0.545	0.643	0.987
BPE Whitespace (bal.)	4.39	0.0233	0.138	0.501	0.683	0.899	0.036	0.409	0.630	0.828

full-versus-smallest (d8-d24) τ is 0.765. Notably, the d16-d24 pair has the lowest τ of the three adjacent pairs, suggesting the danger of using results from smaller language models for larger language model tokenizer development. The ranking changes most at the final doubling of scale, from 596M to the 1.27B target: $\tau = 0.752$ corresponds to about one tokenizer pair in eight ordered differently at the two scales. Rank agreement between adjacent scales does not tighten as scale grows within our range, so a tokenizer selected on val BPB at a smaller scale can sit several rank positions away at the target scale, and a small-scale sweep is a screening tool rather than a substitute for a target-scale comparison.

BLiMP carries a separate cross-scale caveat: under the unnormalized total-log-likelihood scoring described in §C, aggregate BLiMP accuracy is lower for our 1.27B models than for the 300M pilots, driven by a cluster of quantifier subtasks whose accuracy collapses under this scoring at the larger scale. We therefore compare BLiMP only within a single scale.

E.4 Cross-Model Rank Correlation of Code BPB

While the math+code models (§F.3) share tokenizers with the natural-language-focused models, they are trained on very different data mixtures, so code BPB levels are not comparable between the two by construction: for a given tokenizer, a lower value under a math+code model does not mean we should expect lower code BPB under the corresponding natural-language-focused model. Table 14 compares code BPB ranks between the two

Table 11: Intrinsic tokenizer evaluation results computed on a subset of FineWeb-Edu and FineWeb2. Concretely, we sample 1k documents per language for each of the languages present in our training data. Length-unit-dependent metrics use $u = \text{bytes}$, with one exception: Fertility keeps its default $u = \text{words}$ in both this table and Table 10 (Table 1). The value after Fertility’s $\pm$ is the standard deviation of the per-document tokens-per-word ratio across the sampled documents, which Table 10 does not print because its unit of measurement is a single FLORES+ line. Compression is measured in *bytes* per token and is not comparable with the Compression column of Table 10, which uses $u = \text{lines}$. Tokenizers are the same 43 members as Table 10, plus the two off-the-shelf reference tokenizers (marked with †), excluded from every ranking and aggregate statistic except the cross-scale check of Table 17.

Tokenizer	Fertility $\downarrow$	Compression $\uparrow$	Gini $\downarrow$	Vocab CoV $\downarrow$	$\bar{H}_2 \uparrow$	Bigram $\eta \uparrow$	Char Split $\downarrow$
Mistral-Nemo †	4.82 $\pm$ 13.94	4.2326	0.128	0.396	0.572	0.787	0.010
LLaMA-3 †	5.15 $\pm$ 12.40	3.5226	0.161	0.388	0.542	0.734	0.105
BPE Punct (bal.)	4.73 $\pm$ 11.64	3.9624	0.111	0.417	0.533	0.760	0.028
BPE Punct (eng.)	8.90 $\pm$ 26.00	2.4434	0.135	0.534	0.519	0.669	0.226
BPE GPT-4o (bal.)	4.36 $\pm$ 11.29	4.2847	0.132	0.407	0.578	0.801	0.033
BPE GPT-4o NFC	4.36 $\pm$ 11.29	4.2847	0.132	0.407	0.578	0.801	0.033
BPE GPT-4o (eng.)	8.95 $\pm$ 26.20	2.4458	0.140	0.537	0.523	0.678	0.259
BPE GPT-4o (eng., full-byte)	8.95 $\pm$ 26.21	2.4455	0.140	0.537	0.523	0.678	0.259
BPE GPT-4o (code)	6.93 $\pm$ 16.48	2.6289	0.103	0.501	0.538	0.681	0.200
BPE Claude (bal.)	4.42 $\pm$ 11.55	4.2538	0.132	0.414	0.544	0.791	0.032
BPE Claude NFC	4.42 $\pm$ 11.55	4.2538	0.132	0.414	0.544	0.791	0.032
BPE Claude (eng.)	8.89 $\pm$ 26.05	2.4553	0.135	0.540	0.522	0.677	0.246
BPE RightAlign (bal.)	4.36 $\pm$ 11.29	4.2847	0.132	0.405	0.578	0.801	0.033
BPE RightAlign NFC	4.36 $\pm$ 11.29	4.2847	0.132	0.405	0.578	0.801	0.033
Unigram GPT-4o	4.40 $\pm$ 9.11	3.5065	0.175	0.365	0.326	0.684	0.044
Unigram Claude	4.51 $\pm$ 9.64	3.4771	0.173	0.365	0.335	0.679	0.049
Unigram RightAlign	4.40 $\pm$ 9.11	3.5064	0.175	0.365	0.326	0.684	0.044
SuperBPE GPT-4o (bal.)	4.41 $\pm$ 11.46	4.2975	0.128	0.451	0.665	0.818	0.038
PA-BPE NFC GPT-4o	4.20 $\pm$ 12.48	4.8100	0.146	0.259	0.562	0.826	0.017
Unigram GPT-4o (highmid)	4.93 $\pm$ 9.03	3.0457	0.179	0.400	0.338	0.660	0.154
Unigram GPT-4o (highres)	5.89 $\pm$ 11.19	2.6422	0.163	0.516	0.334	0.638	0.200
MinGram SCRIPT-enc	4.33 $\pm$ 11.42	4.3649	0.130	0.403	0.526	0.792	0.000
MinGram SCRIPT-enc (nl-split)	4.33 $\pm$ 11.42	4.3654	0.130	0.403	0.526	0.792	0.000
BPE SCRIPT-enc GPT-4o	4.35 $\pm$ 11.28	4.3009	0.132	0.402	0.572	0.801	0.000
BPE GPT-2 (bal.)	4.70 $\pm$ 11.62	4.0035	0.112	0.411	0.546	0.765	0.029
BPE GPT-4o (all-eq.)	4.18 $\pm$ 12.10	4.7119	0.140	0.359	0.566	0.823	0.017
BPE Claude (all-eq.)	4.24 $\pm$ 12.28	4.6684	0.142	0.360	0.531	0.811	0.016
BPE Punct (all-eq.)	4.62 $\pm$ 12.21	4.2193	0.091	0.383	0.525	0.765	0.015
BPE GPT-4o NFC (all-eq.)	4.18 $\pm$ 12.10	4.7119	0.140	0.359	0.566	0.823	0.017
BPE GPT-4o (all-eq., no-rep.)	4.18 $\pm$ 12.10	4.7119	0.140	0.359	0.566	0.823	0.017
BPE GPT-4o (bal., mf50)	4.36 $\pm$ 11.29	4.2847	0.132	0.407	0.578	0.801	0.033
BPE NFC clean (bal.)	4.42 $\pm$ 11.18	4.2056	0.113	0.413	0.541	0.783	0.033
BPE NFC clean plus2 (bal.)	4.47 $\pm$ 11.64	4.2161	0.132	0.425	0.541	0.783	0.031
BPE NFC clean plus3 (bal.)	4.47 $\pm$ 11.65	4.2127	0.132	0.425	0.541	0.783	0.031
SuperBPE PA-BPE GPT-4o	4.16 $\pm$ 12.35	4.9042	0.145	0.330	0.642	0.846	0.019
SuperBPE PA-BPE GPT-4o t64k	4.14 $\pm$ 11.92	4.8166	0.143	0.383	0.657	0.844	0.020
SuperBPE clean C2 (bal.)	4.31 $\pm$ 12.51	4.6590	0.145	0.373	0.529	0.809	0.018
SuperBPE clean C3 (bal.)	4.19 $\pm$ 12.27	4.8310	0.144	0.345	0.592	0.835	0.019
SuperBPE Punct t64k	4.46 $\pm$ 11.38	4.2271	0.120	0.443	0.626	0.810	0.034
PA-BPE NFC GPT-4o HW	4.12 $\pm$ 12.01	4.8303	0.140	0.287	0.563	0.828	0.016
PA-BPE NFC clean	4.38 $\pm$ 12.98	4.6207	0.146	0.273	0.527	0.801	0.018
Unigram GPT-4o tuned	4.40 $\pm$ 9.21	3.5078	0.171	0.374	0.329	0.690	0.045
BPE GPT-4o (highres)	6.49 $\pm$ 14.16	2.7379	0.183	0.483	0.505	0.677	0.227
BPE GPT-4o (highmid)	5.07 $\pm$ 11.34	3.4759	0.207	0.417	0.504	0.734	0.158
BPE Whitespace (bal.)	4.41 $\pm$ 11.53	4.2929	0.130	0.406	0.696	0.815	0.037

model families for the 25 tokenizers that have a model in both: because rank transfer is a per-tokenizer comparison rather than a correlation panel, this set keeps every eligible tokenizer, including family variants that the primary panel collapses to one representative, both English-only GPT-4o variants (with and without the full byte alphabet, §C.1.1), and the whitespace minimal-structure probe as a labeled control. The rank correlation between the natural-language models’ code BPB rank and the math+code models’ code BPB rank is Spearman $\rho = 0.307$ (95% bootstrap CI -0.191 to 0.715 , $n = 25$, with the labeled control included) and $\rho = 0.359$ (95% CI -0.147 to 0.764 , $n = 24$) with it excluded. Both confidence intervals span zero, so the natural-language models’ code BPB ranking does not transfer detectably to the math+code models’ code BPB ranking. We took this result as further justification of the choice to train dedicated math and code models.

Table 12: Evaluation results for the 1.27B models: 44 tokenizers trained on the standard corpus mixture at a $\sim 128K$ vocabulary, plus two off-the-shelf reference tokenizers (marked with †) excluded from every ranking and aggregate statistic except the cross-scale check of Table 17. Panel membership per row is in Table 5; the rows outside every panel are the two defective English-only retrains, the file-cap null control and the two minimal-structure controls. Lower is better for the BPB columns; higher is better for BLiMP and MultiBLiMP. Code BPB for the 25 tokenizers in the cross-model comparison appears in Table 14, where the two model families’ ranks, not absolute scores, are compared. ‘-’: not evaluated for that tokenizer.

Tokenizer	Perplexity $\downarrow$ (mean cross-language σ)					Ling. $\uparrow$	
	Val	FLORES (tr.)	σ	FLORES (all)	σ	BLiMP	MultiBLiMP
Mistral-Nemo †	0.7212	1.173	0.299	2.756	1.012	0.817	0.915
LLaMA-3 †	0.7198	1.173	0.293	2.662	1.014	0.826	0.914
BPE Punct (bal.)	0.7185	1.167	0.291	2.647	1.018	0.817	0.913
BPE Punct (eng.)	0.7360	1.191	0.308	2.636	1.015	0.807	0.911
BPE GPT-4o (bal.)	0.7131	1.161	0.294	2.641	1.020	0.820	0.916
BPE GPT-4o NFC	0.7133	1.159	0.298	2.638	1.019	0.820	0.920
BPE GPT-4o (eng.)	0.7375	1.197	0.312	2.638	1.015	0.807	0.911
BPE GPT-4o (eng., full-byte)	0.7382	1.201	0.316	2.644	1.017	0.817	0.912
BPE GPT-4o (code)	0.7252	1.179	0.295	2.626	1.009	0.819	0.913
BPE Claude (bal.)	0.7156	1.162	0.296	2.642	1.016	0.816	0.914
BPE Claude NFC	0.7152	1.159	0.300	2.642	1.018	0.819	0.911
BPE Claude (eng.)	0.7330	1.194	0.311	2.641	1.021	0.817	0.907
BPE RightAlign (bal.)	0.7138	1.164	0.293	2.641	1.016	0.813	0.909
BPE RightAlign NFC	0.7138	1.161	0.298	2.642	1.018	0.816	0.911
Unigram GPT-4o	0.7338	1.193	0.304	2.687	1.010	0.834	0.903
Unigram Claude	0.7359	1.189	0.302	2.683	1.007	0.841	0.905
Unigram RightAlign	0.7338	1.193	0.302	2.684	1.002	0.828	0.908
SuperBPE GPT-4o (bal.)	0.7236	1.173	0.297	2.639	1.013	0.804	0.909
PA-BPE NFC GPT-4o	0.7255	1.186	0.300	2.662	1.006	0.817	0.904
Unigram GPT-4o (highmid)	0.7345	1.197	0.306	2.675	1.003	0.831	0.913
Unigram GPT-4o (highres)	0.7305	1.197	0.305	2.666	0.994	0.824	0.907
MinGram SCRIPT-enc	0.7129	1.165	0.303	2.685	1.048	0.813	0.920
MinGram SCRIPT-enc (nl-split)	0.7134	1.159	0.301	2.671	1.058	0.815	0.918
BPE SCRIPT-enc GPT-4o	0.7128	1.157	0.297	2.657	1.046	0.822	0.916
BPE GPT-2 (bal.)	0.7128	1.157	0.287	2.635	1.016	0.816	0.910
BPE GPT-4o (all-eq.)	0.7177	1.172	0.292	2.652	1.016	0.819	0.918
BPE Claude (all-eq.)	0.7200	1.170	0.291	2.654	1.015	0.817	0.921
BPE Punct (all-eq.)	0.7223	1.168	0.294	2.648	1.019	0.818	0.917
BPE GPT-4o NFC (all-eq.)	0.7165	1.164	0.295	2.646	1.014	0.823	0.917
BPE GPT-4o (all-eq., no-rep.)	0.7165	1.168	0.292	2.648	1.015	0.819	0.913
BPE GPT-4o (bal., mf50)	0.7116	1.158	0.292	2.629	1.013	0.822	-
BPE NFC clean (bal.)	0.7159	1.157	0.296	2.648	1.022	0.821	0.910
BPE NFC clean plus2 (bal.)	0.7162	1.158	0.297	2.638	1.017	0.814	0.918
BPE NFC clean plus3 (bal.)	0.7149	1.155	0.297	2.635	1.017	0.818	0.918
SuperBPE PA-BPE GPT-4o	0.7292	1.181	0.294	2.653	1.008	0.801	0.916
SuperBPE PA-BPE GPT-4o t64k	0.7293	1.181	0.295	2.652	1.011	0.792	0.920
SuperBPE clean C2 (bal.)	0.7285	1.169	0.293	2.650	1.011	0.811	0.912
SuperBPE clean C3 (bal.)	0.7296	1.174	0.294	2.644	1.006	0.803	0.919
SuperBPE Punct t64k	0.7259	1.167	0.293	2.630	1.009	0.808	0.915
PA-BPE NFC GPT-4o HW	0.7194	1.177	0.299	2.656	1.007	0.816	0.914
PA-BPE NFC clean	0.7235	1.178	0.302	2.653	1.007	0.819	0.907
Unigram GPT-4o tuned	0.7333	1.192	0.302	2.686	1.009	0.838	0.912
BPE GPT-4o (highres)	0.7127	1.182	0.294	2.618	0.999	0.814	0.912
BPE GPT-4o (highmid)	0.7137	1.174	0.296	2.637	1.016	0.815	0.910
BPE Whitespace (bal.)	0.7207	1.165	0.294	2.622	1.003	0.814	0.915
Unigram Whitespace tuned (bal.)	0.8401	1.176	0.298	2.576	0.949	0.805	0.918

F Additional Analyses

F.1 Held-Out Prediction of the Reference Tokenizers

We test how far a fit estimated on the trained tokenizer panel predicts a tokenizer the panel never saw. For each of four targets, val BPB and FLORES trained-31 BPB (fit on the 39-tokenizer extended panel of §§ C.1.1 and 4.1), and code BPB and MBPP pass@1 from the math+code models (both fit on the 20-tokenizer math+code grid of Table 18, excluding the two references), we take the best univariate intrinsic predictor and a multivariate fit on up to 3 standardized predictors selected by leave-one-family-out cross-validation inside the panel (tokenizers from the same family, e.g., a base tokenizer and a preset variant, are held out

Table 13: Pilot-scale (300M) evaluation results; lower is better for all metrics. [†]: off-the-shelf reference tokenizer, as in Table 12. The roster is the 30 tokenizers of Table 12 that also have a model at this scale, plus the same 2 references; the remaining configurations were added after the pilot sweep and have no 300M model.

Tokenizer	Val BPB	FLORES (tr.)	FLORES (all)	Code BPB
Mistral-Nemo [†]	0.8210	1.292	2.827	0.612
LLaMA-3 [†]	0.8193	1.304	2.751	0.622
BPE Punct (bal.)	0.8179	1.295	2.737	0.613
BPE Punct (eng.)	0.8334	1.343	2.748	0.646
BPE GPT-4o (bal.)	0.8067	1.290	2.734	0.599
BPE GPT-4o NFC	0.8064	1.287	2.740	0.599
BPE GPT-4o (eng.)	0.8327	1.344	2.749	0.648
BPE GPT-4o (code)	0.8282	1.332	2.737	0.625
BPE Claude (bal.)	0.8131	1.291	2.740	0.599
BPE Claude NFC	0.8138	1.288	2.736	0.602
BPE Claude (eng.)	0.8327	1.343	2.748	0.639
BPE RightAlign (bal.)	0.8066	1.289	2.736	0.599
BPE RightAlign NFC	0.8065	1.286	2.734	0.599
Unigram GPT-4o	0.8407	1.328	2.786	0.642
Unigram Claude	0.8471	1.334	2.791	0.643
Unigram RightAlign	0.8408	1.326	2.790	0.642
PA-BPE NFC GPT-4o	0.8333	1.299	2.760	0.624
PA-BPE NFC clean	0.8309	1.294	2.752	0.614
BPE Whitespace (bal.)	0.8101	1.297	2.725	0.603
BPE GPT-2 (bal.)	0.8083	1.292	2.730	0.596
BPE GPT-4o (highres)	0.8084	1.323	2.721	0.603
BPE GPT-4o (highmid)	0.8053	1.300	2.728	0.598
Unigram GPT-4o tuned	0.8440	1.335	2.801	0.644
Unigram Whitespace tuned (bal.)	0.9633	1.312	2.692	0.720
PA-BPE NFC GPT-4o HW	0.8165	1.280	2.737	0.606
SuperBPE Punct t64k	0.8127	1.298	2.720	0.594
SuperBPE GPT-4o (bal.)	0.8098	1.302	2.729	0.592
SuperBPE PA-BPE GPT-4o	0.8214	1.291	2.742	0.607
SuperBPE PA-BPE GPT-4o t64k	0.8191	1.293	2.742	0.600
SuperBPE clean C2 (bal.)	0.8262	1.279	2.734	0.607
SuperBPE clean C3 (bal.)	0.8214	1.283	2.737	0.609
BPE NFC clean (bal.)	0.8165	1.286	2.740	0.605

together, so no fold trains on a tokenizer from its test point’s family), then predict the two off-the-shelf reference tokenizers’ models, Mistral-Nemo and LLaMA-3, which share architecture and training data with the panel but were excluded from every panel fit. We report each reference’s prediction error standardized by the fit’s cross-validated root-mean-squared error (CV-RMSE), and whether the observed value falls inside the CV-RMSE-calibrated 90% prediction interval (a closed-form t -interval, not a bootstrap interval).

Table 15 reports all eight fits. Under the best univariate fits, 6 of the 8 reference-target predictions (4 targets times 2 references) fall inside their 90% prediction interval; both exceptions are LLaMA-3, on FLORES (trained) BPB at +2.10 standardized errors and on the math+code models’ code BPB at −2.01. The multivariate fits have a higher in-sample R^2 and a lower CV-RMSE than the univariate fits on every target, yet 4 of their 8 reference predictions fall outside the 90% interval, against 2 of 8 for the univariate fits: the narrower multivariate intervals overstate the precision available for a tokenizer from outside the panel.

The selected predictors differ by training mixture. The natural-language models’ val BPB and FLORES (trained) BPB are best predicted by tokens per identifier and identifier fragmentation; the math+code models’ code BPB is best predicted by the Gini coefficient measured on FineWeb, and none of its selected predictors is an entropy metric (unigram, bigram, or trigram entropy, or Rényi efficiency), although those metrics correlate significantly with the natural-language models’ code BPB (§F.3). Table 16 shows the same Gini coefficient

Table 14: Code BPB and code BPB rank for the natural-language models and the math+code models, for the 25 tokenizers with a model in both families, sorted by natural-language-model rank. Code BPB levels are not comparable across the two columns; only the rank columns are. [†]: labeled control (whitespace minimal-structure probe), excluded from the without-control correlation reported in the text. *: this row’s math+code model was trained before a change to the cluster software environment that the other 24 postdate. It is kept because it is the only math+code run for that tokenizer.

Tokenizer	Std-1B BPB	Std-1B rank	Math+code BPB	Math+code rank
MinGram SCRIPT-enc	0.5125	1.0	0.3793	9.0
BPE SCRIPT-enc GPT-4o	0.5186	2.0	0.3781	6.0
BPE Claude (bal.)	0.5203	3.0	0.3861	18.0
BPE GPT-4o (bal.)*	0.5206	4.0	0.3773	5.0
BPE Claude NFC	0.5206	5.0	0.3845	17.0
BPE GPT-4o NFC	0.5207	6.0	0.3786	8.0
BPE RightAlign (bal.)	0.5223	7.0	0.3798	10.0
BPE RightAlign NFC	0.5225	8.0	0.3809	13.0
BPE NFC clean (bal.)	0.5226	9.0	0.3815	14.0
BPE NFC clean plus3 (bal.)	0.5232	10.0	0.3834	16.0
BPE NFC clean plus2 (bal.)	0.5249	11.0	0.3785	7.0
SuperBPE GPT-4o (bal.)	0.5249	12.0	0.3834	15.0
SuperBPE clean C2 (bal.)	0.5259	13.0	0.3906	21.0
BPE Whitespace (bal.) [†]	0.5269	14.0	0.3742	4.0
BPE Punct (bal.)	0.5302	15.0	0.3801	11.0
SuperBPE clean C3 (bal.)	0.5306	16.0	0.3896	20.0
PA-BPE NFC GPT-4o	0.5335	17.0	0.3878	19.0
BPE Claude (eng.)	0.5426	18.0	0.3652	1.0
BPE GPT-4o (code)	0.5447	19.0	0.3805	12.0
BPE Punct (eng.)	0.5551	20.0	0.3724	3.0
Unigram RightAlign	0.5566	21.0	0.3989	22.0
BPE GPT-4o (eng.)	0.5567	22.0	0.3709	2.0
BPE GPT-4o (eng., full-byte)	0.5568	23.0	0.3991	23.0
Unigram GPT-4o	0.5569	24.0	0.3993	24.0
Unigram Claude	0.5578	25.0	0.4032	25.0

association with the math+code models’ code BPB only under FineWeb measurement, not under FLORES+ measurement.

F.2 Intrinsic–Downstream Correlations Robustness Checks

Table 16 repeats the aggregate correlation analysis of Table 2, with the FLORES+-measured intrinsic metrics replaced by their FineWeb-Edu/FineWeb2-measured counterparts, over the same primary panel and downstream targets. Four rows (identifier fragmentation, digit boundary F_1 , operator isolation on the math corpus, and AST alignment) are byte-identical between the two tables, since those metrics are computed on code and math corpora rather than on the natural-language corpus that varies between them; see the caption below.

As a robustness check, we repeat the correlation analysis at the extended-panel level (§4.3): each of the 31 tokenizer families in the 39-tokenizer extended panel is collapsed to its family-mean score before computing the Spearman correlation, so same-family tokenizers cannot count as independent evidence, and significance is assessed by a permutation test that reshuffles whole families rather than individual tokenizers. This extended-level correction is computed over a larger set of comparisons than the 98 cells printed in Table 2, so its adjusted p -values are not directly comparable to the table’s. Under this check, the Rényi-efficiency and digit-boundary- F_1 correlations reported above both remain significant ($p_{\text{adj}} = 0.016$ and 0.029), as do four of the five information-theoretic correlations with FLORES(trained) BPB; the fifth, unigram entropy, falls just short of significance ($p_{\text{adj}} = 0.062$). We checked whether any single tokenizer drives the four correlations discussed above (Rényi efficiency with FLORES(trained) BPB, digit boundary F_1 with BLiMP, digit boundary F_1 with Code BPB, and AST alignment with MBPP) by removing tokenizers one at a time from the relevant panel and recomputing each correlation. Every one of these leave-one-out recomputations

Table 15: Held-out prediction of the two off-the-shelf reference tokenizers. For each downstream target, the best univariate intrinsic predictor and the multivariate fit (up to 3 standardized predictors, selected by leave-one-family-out cross-validation), with the in-sample R^2 , the cross-validated root-mean-squared error (CV-RMSE), and each reference’s prediction error divided by the CV-RMSE (standardized error). [†]: the observed value falls outside the fit’s 90% prediction interval. Intrinsic predictors are measured on FLORES+ unless marked otherwise. Val BPB and FLORES (trained) BPB are fit on the 39-tokenizer extended std-1B panel; the two math+code targets are fit on the 20-tokenizer math+code panel.

Target	Fit	Predictors	R^2	CV-RMSE	Standardized error	
					Mistral-Nemo	LLaMA-3
Val BPB ($n=39$)	univ.	tokens per identifier	0.53	0.0061	+0.48	+1.43
	multiv.	fertility, Gini coefficient, tokens per identifier	0.68	0.0054	+0.35	+1.84 [†]
FLORES (trained) BPB ($n=39$)	univ.	identifier fragmentation	0.59	0.0091	+0.36	+2.10 [†]
	multiv.	fertility, identifier fragmentation, indentation consistency	0.81	0.0062	+0.41	+2.18 [†]
Code BPB (math+code) ($n=20$)	univ.	Gini coefficient (FineWeb)	0.48	0.0074	−0.40	−2.01 [†]
	multiv.	vocabulary utilization (FineWeb), trigram entropy, operator isolation (code corpus)	0.68	0.0070	−0.05	−0.90
MBPP pass@1 (math+code) ($n=20$)	univ.	AST boundary alignment	0.26	0.0793	−1.15	−0.03
	multiv.	token length, unigram entropy, AST boundary alignment	0.50	0.0739	−1.78 [†]	+1.77 [†]

keeps the original sign and $p < 0.05$ before correction for multiple testing: this holds across all 29 possible single-tokenizer removals for the two correlations computed on the full primary panel (Rényi efficiency/FLORES and digit boundary F_1 /BLiMP), and across all 19 possible removals for the two correlations restricted to primary-panel tokenizers with a math+code model (digit boundary F_1 /Code BPB and AST alignment/MBPP); the worst-case p -value across every removal of all four correlations is 0.0162. Under the correction applied jointly across the full table (Table 2), removing a single tokenizer, Unigram Claude (Claude pretokenizer, UnigramLM algorithm), is enough to push three of these four correlations, digit boundary F_1 with BLiMP, digit boundary F_1 with Code BPB and AST alignment with MBPP, just above the corrected significance threshold, and 12 of the 29 possible single removals move at least one of the four.

We also checked robustness to removing whole groups of tokenizers that share a non-default value on one design axis (algorithm, pretokenization, or training-data composition; §4.1), keeping only the tokenizers that share this panel’s single most common value on that axis. These axis-based removals affect the four correlations less uniformly than single-tokenizer removal: three weaken but keep their sign, while the AST-alignment/MBPP correlation reverses sign, becoming small and not significant ($\rho = -0.25$), when restricted to the 7 tokenizers in this panel that use the GPT-4o pretokenizer, this panel’s most common pretokenization choice. This reversal is consistent with the line-break attribution below: the AST-alignment/MBPP association reflects a difference between pretokenizer choices rather than a relationship that holds within a single pretokenizer choice.

Table 16: Aggregate Spearman (FineWeb-2/-Edu intrinsic) ρ between intrinsic metrics and downstream performance ($n = 29$ primary-panel tokenizers; $n = 19$ for the math+code Code BPB and MBPP columns, the primary-panel members that also have a math+code model). Intrinsic metrics are measured on FineWeb-Edu and FineWeb2 with length unit $u = \text{bytes}$, so unit-dependent metrics (compression rate, fertility) are not comparable with the same column name in Table 2, which uses $u = \text{lines}$. Four rows, Ident. fragmentation, Digit boundary F1, Operator isolation (math), and AST alignment, are computed on code and math corpora rather than on FineWeb, so they are byte-identical to the same rows of Table 2. **Bold** indicates significance, with stars marking the (BH-adjusted) thresholds, over the 98 printed cells and exactly as in Table 2: * $p_{\text{adj}} < 0.05$, ** $p_{\text{adj}} < 0.01$, *** $p_{\text{adj}} < 0.001$.

Intrinsic Metric	Val BPB	FLORES		Math+code		BLiMP	MultiBLiMP
		trained	all	Code BPB	MBPP		
Digit boundary F1	-0.42	-0.41	-0.67**	-0.62*	0.21	-0.51*	0.32
Ident. fragmentation	0.53*	0.57*	0.36	0.41	-0.21	0.37	-0.45
Rényi eff. ($\alpha=2$)	-0.53*	-0.72***	-0.42	-0.49	0.12	-0.25	0.32
Operator isolation (math)	0.54*	0.38	0.55*	0.39	0.27	0.32	-0.25
Gini coefficient	0.33	0.62**	0.38	0.60*	-0.10	0.24	-0.36
UTF-8 char split	0.38	0.56*	-0.33	0.25	-0.26	0.09	-0.43
Vocab utilization	0.26	0.07	0.70**	0.54	0.05	0.32	-0.17
Trigram entropy	-0.37	-0.54*	0.12	-0.28	0.08	-0.14	0.33
UTF-8 boundary crossing	0.18	0.34	-0.33	0.20	-0.27	0.18	-0.33
Fertility	0.27	0.41	-0.37	0.16	0.23	-0.04	-0.28
Compression rate	-0.32	-0.47	0.24	-0.19	-0.03	-0.17	0.28
Bigram entropy	-0.35	-0.53*	0.00	-0.21	-0.02	-0.23	0.30
AST alignment	0.31	0.01	0.36	0.16	0.61*	0.10	-0.05
Unigram entropy	-0.34	-0.51*	0.04	-0.12	0.04	-0.22	0.26

Table 17: Kendall’s τ between per-tokenizer val BPB rank vectors across the four training scales, 18-tokenizer panel ($n = 18$ for every cell; unlike the other tables in this paper, this panel includes the two off-the-shelf reference tokenizers). All models at each scale were trained under one software configuration for that scale.

	d8 (tiny)	d12 (small)	d16 (pilot)	d24 (full)
d8 (tiny)	1.000	0.882	0.882	0.765
d12 (small)	0.882	1.000	0.922	0.725
d16 (pilot)	0.882	0.922	1.000	0.752
d24 (full)	0.765	0.725	0.752	1.000

F.3 Math and Code Generation (math+code Models)

Table 18 reports the math+code models’ generation benchmarks and code BPB. Under identical training data, architecture, and budget, MBPP pass@1 ranges from 0.000 to 0.250 across the 20 custom tokenizers, while HumanEval pass@1 stays within 0.134 to 0.177. The paragraphs below trace the MBPP spread to the vocabularies’ line-break handling, and then examine how the code-structure metrics of §3 relate to it.

Line-break handling and code generation. Pretokenizers differ in whether the tokenizer may learn tokens that fuse punctuation with a following line break. The GPT-4o regex permits such merges: the balanced GPT-4o BPE vocabulary contains 1,024 fused punctuation-line-break tokens, for example `) \n` and `;\n`, among 1,091 tokens containing a line break overall. The punct, Claude, and clean-multi regexes split before a line break, so their vocabularies contain none. This vocabulary property predicts code generation. Across the 18 custom models with byte-level vocabularies (token-byte counts are comparable only within this subset; the two SCRIPT-encoding members and the two references are excluded), the number of vocabulary tokens containing a line break correlates with MBPP pass@1 at Spearman $\rho = -0.717$ (significant, $p = 8.1 \times 10^{-4}$), and models whose tokenizer has at least one fused punctuation-line-break token average MBPP pass@1 0.075, against 0.226 for models whose tokenizer has none (Mann–Whitney $p = 4.5 \times 10^{-4}$). The difference is visible in

Table 18: Evaluation results for the math+code models: 20 LMs trained from scratch on a math and code mixture (§4.2) using custom tokenizers at a $\sim 128\text{K}$ vocabulary, plus two off-the-shelf reference tokenizers (marked with [†]) excluded from every ranking (§4.1). ^b: this row’s tokenizer cannot represent all 256 byte values (§C.1.1). Both marked rows are English-only builds kept in this panel with the defect flagged rather than dropped, because no defect-free tokenizer for either configuration has a math+code run at this scale; the GPT-4o English build, which does have a defect-free counterpart, is represented here by that counterpart instead. Lower is better for Code BPB; higher is better for MBPP, HumanEval, and GSM8K.

Tokenizer	MBPP	HumanEval	GSM8K	Code BPB
BPE Punct (bal.)	0.230	0.171	0.189	0.380
BPE Punct (eng.)	0.228	0.165	0.208	0.372
BPE GPT-4o (bal.)	0.174	0.165	0.198	0.377
BPE GPT-4o NFC	0.120	0.152	0.208	0.379
BPE GPT-4o (eng., full-byte)	0.064	0.152	0.218	0.399
BPE GPT-4o (code)	0.016	0.159	0.216	0.380
BPE Claude (bal.)	0.250	0.171	0.227	0.386
BPE Claude NFC	0.224	0.159	0.230	0.385
BPE Claude (eng.) ^b	0.234	0.165	0.221	0.365
BPE RightAlign (bal.)	0.072	0.152	0.234	0.380
BPE RightAlign NFC	0.104	0.171	0.224	0.381
Unigram GPT-4o	0.000	0.134	0.187	0.399
Unigram Claude	0.222	0.146	0.165	0.403
Unigram RightAlign	0.000	0.140	0.175	0.399
SuperBPE GPT-4o (bal.)	0.068	0.159	0.180	0.383
PA-BPE NFC GPT-4o	0.130	0.165	0.203	0.388
MinGram SCRIPT-enc	0.016	0.177	0.199	0.379
BPE SCRIPT-enc GPT-4o	0.136	0.171	0.205	0.378
BPE NFC clean (bal.)	0.218	0.159	0.177	0.381
SuperBPE clean C2 (bal.)	0.202	0.134	0.241	0.391
Mistral-Nemo [†]	0.000	0.152	0.184	0.378
LLaMA-3 [†]	0.082	0.171	0.233	0.378

the generated text itself: the fraction of a model’s MBPP generations containing a line break correlates with its MBPP pass@1 at $\rho = +0.593$ (significant, $p = 5.8 \times 10^{-3}$; this measurement uses generated text rather than token bytes, so it covers all 20 custom models). Models trained with vocabularies containing many fused punctuation-line-break tokens emit fewer line breaks in their generations, and MBPP scores Python, in which line breaks are syntactic.

Code-structure metrics in the math+code models. In the math+code models, AST boundary alignment correlates with MBPP pass@1 at $\rho = +0.65$ (significant, $p_{\text{adj}} = 0.036$, $n = 20$; 38 tests in one Benjamini-Hochberg family), the only significant code-structure correlation on this roster (the 20 custom tokenizers of Table 18, no references). We do not interpret this correlation as an effect of AST boundary alignment itself, for two reasons. First, this association is not separable from the line-break handling described above: controlling for the number of vocabulary tokens containing a line break reduces it to $\rho = +0.26$ (not significant, $p = 0.31$, $n = 18$, this panel’s members with byte-level vocabularies), consistent with the pretokenizer-axis robustness check in §5, where the same correlation reverses sign once restricted to tokenizers sharing one pretokenizer choice. Second, the two MinGram tokenizers differ only in whether merges may cross line breaks (Table 5). MBPP pass@1 is 0.016 for the arm whose merges may cross them and 0.216 for the arm that splits there, 2.3 panel standard deviations apart, while their AST boundary alignment differs by 0.0014 standard deviations, too small a difference to explain that gap. The splitting arm is a member of the extended math+code roster rather than the 20-member panel, so it has no row in Table 18. At the extended level (the 24-custom-tokenizer extended math+code roster, every family variant rather than one representative per family), the same association is $\rho = +0.66$

(significant, $p_{\text{adj}} = 0.017$, $n = 24$), the same direction and significance as on the primary roster above. Which metrics predict code BPB also changes with the training mixture. Under the natural-language models, four entropy metrics correlate significantly with code BPB (bigram entropy $\rho = -0.55$, trigram entropy -0.64 , unigram entropy -0.48 , Rényi efficiency -0.69 , all $p_{\text{adj}} < 0.05$, $n = 29$); under the math+code models, none of the four is significant (Table 2). The natural-language models’ code BPB is a downstream column of the released correlation artifact that the paper’s tables do not print, and it is corrected within that artifact’s own family (§C.1.2); the same holds for the line-break counts above. A metric’s predictive value is therefore not a property of the metric alone but of the metric paired with a training distribution.

Operator isolation. Operator isolation, whether operators are dedicated tokens rather than fused with adjacent operands (§3), is measured separately on the prose, math, and code corpora. Its correlation with the math+code models’ MBPP pass@1 is positive in all three measurement domains ($\rho = 0.52$ prose, 0.34 code, 0.28 math, $n = 20$ custom tokenizers) but none survives BH correction (smallest $p_{\text{adj}} = 0.058$, prose corpus). The panel contains two UnigramLM tokenizers with MBPP pass@1 of exactly 0.000; removing them makes all three correlations significant ($p_{\text{adj}} = 0.012, 0.007, 0.007$; $n = 18$). Because significance depends on those two models, we read the association as suggestive rather than established; and because the tokenizers that differ in operator isolation also differ in their whole pretokenization regex, the association cannot be attributed to operator handling specifically.

F.4 External-Model Validation of Per-Example Tokenization Metrics

As a check, we test whether per-example code-tokenization quality predicts per-problem generation success on publicly released code models trained independently of this paper’s pipeline.

We evaluate 5 publicly released code models (StarCoder2-3B, Qwen2.5-Coder-3B, CodeGemma-2B, phi-2, stable-code-3b), each with its own tokenizer, spanning 5 distinct tokenizer classes and vocabularies from 49,152 to 256,000 tokens, on HumanEval (164 problems) and MBPP (500 problems).

Design. For each (model, problem) pair we compute per-example AST-boundary and identifier-fragmentation metrics, under that model’s own tokenizer, on the prompt and on the reference solution, and join them to the binary pass/fail outcome. The primary estimator is a conditional logistic regression stratified on problem, which conditions out item difficulty by construction, with one ability term per model; a Bayesian generalized linear mixed model with a random intercept per problem serves as a companion specification. Significance uses Benjamini-Hochberg FDR correction within each text-basis-by-task metric family.

Result: null on both panels. No metric is FDR-significant within either task on the external 5-model panel: the smallest within-task BH-adjusted p over both tasks and both text bases is 0.219 (target text basis, HumanEval, end-alignment rate). Applying the identical design to this paper’s own 20 custom math+code tokenizers (the math+code panel of Table 18), joined to their own MBPP and HumanEval generations, gives the same null result: the smallest within-task BH-adjusted p is 0.087 (target text basis, MBPP, identifier fragmentation rate), and 0 of 53 fitted pooled-and-within-task cells are FDR-significant.

A specification artifact in the pooled fit. Pooling HumanEval and MBPP rows into one stratified fit requires one ability term per model per task; fitting a single ability term per model shared across both tasks instead produces an association in the pooled fit that is absent from both tasks individually. On the internal 20-tokenizer panel, a likelihood-ratio test rejects the shared-ability constraint for every one of the 18 pooled-scope metric cells, significant in every case (statistic 137.8 to 193.5, 20 degrees of freedom). The spurious pooled association is therefore a specification artifact of sharing one ability vector across two tasks with different pass rates, not a tokenization effect, and we do not report pooled-scope results as findings on either panel.